Funções de Caracteres

- **INSTR**(x, localizar_string):
  - ***Procura*** o ***string*** em "x", ***retornando*** a ***posição***
- ***Exemplo (01):***
  - ***Obtém*** a ***posição*** onde o ***string*** "*Science*" ***ocorre*** na ***coluna*** **NM_PRODUTO** para o **produto 1**

```
SELECT nm_produto, INSTR(nm_produto, 'Science')
FROM tb_produtos
WHERE id_produto = 1;
```

NM_PRODUTO	INSTR(NM_PRODUTO,'SCIENCE')
Modern Science	8

Funções de Caracteres

- **INSTR**(x, localizar_string):
  - ***Procura*** o ***string*** em "x", ***retornando*** a ***posição***
- ***Exemplo (02):***
  - ***Exibe*** a ***posição*** onde a ***2ª ocorrência*** do ***caractere*** "**e**" ***ocorre***, ***começando*** no ***início*** do **NM_PRODUTO**

```
SELECT nm_produto, INSTR(nm_produto, 'e', 1, 2)
FROM tb_produtos
WHERE id_produto = 1;
```

NM_PRODUTO	INSTR(NM_PRODUTO,'E',1,2)
Modern Science	11

Funções de Caracteres

- **LENGTH(x):**
  - ***Obtém*** o ***número*** de ***caracteres*** em "x"
- ***Exemplo:***
  - ***Obtém*** o ***comprimento*** dos ***strings*** da ***coluna*** **NM_PRODUTO** da tabela **TB_PRODUTOS**

```
SELECT nm_produto, LENGTH(nm_produto)
FROM tb_produtos;
```

NM_PRODUTO	LENGTH(NM_PRODUTO)
Z Files	7
2412: The Return	16
Space Force 9	13
From Another Planet	19
Classical Music	15
Pop 3	5
Creative Yell	13
My Front Line	13
Modern Science	14
Chemistry	9
Supernova	9
Tank War	8

Funções de Caracteres

- **LOWER(x):**
  - ***Converte*** as ***letras*** de "x" para ***minúsculo***
- **UPPER(x):**
  - ***Converte*** as ***letras*** de "x" para ***maiúsculo***
- ***Exemplo:***
  - ***Converte*** os ***strings*** da ***coluna*** **NOME** para ***maiúsculo*** e os ***strings*** da ***coluna*** **SOBRENOME** em ***minúsculo***

```
SELECT UPPER(nome), LOWER(sobrenome)
FROM tb_funcionarios;
```

Funções de Caracteres

- **LPAD**(x, largura, string_preenchimento):
  - ***Preenche*** "x" com ***espaços*** à ***esquerda*** até ***atingir*** o ***comprimento total*** do ***string*** (***largura***)
  - ***Permite fornecer*** o ***string*** de ***preenchimento*** (***opcional***)

- **RPAD**(x, largura, string_preenchimento):
  - ***Preenche*** "x" com ***espaços*** à ***direita*** até ***atingir*** o ***comprimento total*** do ***string*** (***largura***)
  - ***Permite fornecer*** o ***string*** de ***preenchimento*** (***opcional***)

Funções de Caracteres

- **LPAD** e **RPAD**:
- ***Exemplo:***
  - A ***coluna*** **NM_PRODUTO** é ***preenchida*** à **DIREITA** (***comprimento igual*** a **30** – ***string ponto*** (**.**))
  - A ***coluna*** **PRECO** é ***preenchida*** à **ESQUERDA** (***comprimento igual*** a **8** – ***string*** ***+**)

```
SELECT RPAD(nm_produto, 30, '.'), LPAD(preco, 8, '*+')
FROM tb_produtos
WHERE id_produto < 4;
```

RPAD(NM_PRODUTO,30,'.')	LPAD(PRECO,8,'*+')
Modern Science.....	*+*19.95
Chemistry.....	*+*+*+30
Supernova.....	*+*25.99

Funções de Caracteres

- **LTRIM**(x, string_corte):
  - ***Corta*** a ***esquerda*** de "x"
  - ***Permite fornecer*** o ***string*** de ***corte*** (***opcional***)
  - ***Caso não*** seja ***informado*** o ***string*** de ***corte***, os ***espaços*** serão ***cortados*** por ***padrão***
- **RTRIM**(x, string_corte):
  - ***Corta*** a ***direita*** de "x"
  - ***Permite fornecer*** o ***string*** de ***corte*** (***opcional***)
  - ***Caso não*** seja ***informado*** o ***string*** de ***corte***, os ***espaços*** serão ***cortados*** por ***padrão***

Funções de Caracteres

- **TRIM**(x, string_corte):
  - ***Corta*** a ***esquerda*** e a ***direita*** de "x"
  - ***Permite fornecer*** o ***string*** de ***corte*** (***opcional***)
  - ***Caso não*** seja ***informado*** o ***string*** de ***corte***, os ***espaços*** serão ***cortados*** por ***padrão***
- ***Exemplo:***

```
SELECT
   LTRIM('    Olá pessoal tudo joia?'),
   RTRIM('Oi tudo bem!abcabc', 'abc'),
   TRIM('0' FROM '000Treinamento em Oracle!00000')
FROM dual;
```

LTRIM('OLÁPESSOALTUDOJÓIA?')	RTRIM('OITUDOBEM!ABCABC','ABC')	TRIM('0'FROM'000TREINAMENTOEMORACLE!00000')
Olá pessoal tudo jóia?	Oi tudo bem!	Treinamento em Oracle!

Funções de Caracteres

- **NVL**(x, valor):
  - ***Retorna*** um ***valor caso*** "x" seja **NULO**, caso ***contrário***, "x" é ***retornado***
- ***Exemplo:***
  - ***Recupera*** as ***colunas*** **ID_CLIENTE** e **TELEFONE** da **TB_CLIENTES**
  - Os ***valores NULOS*** da ***coluna*** **TELEFONE** serão ***convertidos*** para o ***string*** "Telefone inexistente"

```
SELECT id_cliente, NVL(telefone, 'Telefone inexistente')
FROM tb_clientes
ORDER BY id_cliente DESC;
```

ID_CLIENTE	NVL(TELEFONE,'TELEFONEINEXISTENTE')
6	800-555-1215
5	Telefone inexistente
4	800-555-1214
3	800-555-1213
2	800-555-1212
1	800-555-1211

Funções de Caracteres

- **NVL2**(x, valor1, valor2):
  - ***Retorna*** valor1 se "x" ***não*** for **NULO**, caso ***contrário***, valor2 é ***retornado***
- ***Exemplo:***
  - ***Recupera*** as ***colunas*** **ID_CLIENTE** e **TELEFONE** da **TB_CLIENTES**
  - Os ***valores*** da ***coluna*** **TELEFONE** são ***convertidos*** para o ***string*** "Telefone existente" e os ***valores*** **NULOS** são ***convertidos*** em "Telefone inexistente"

```
SELECT id_cliente, NVL2(telefone, 'Telefone existente', 'Telefone inexistente')
FROM tb_clientes
ORDER BY id_cliente DESC;
```

Funções de Caracteres

- **REPLACE**(x, string_busca, string_substituta):
  - ***Procura*** string_busca em "x" e ***substitui*** por string_substituta
- ***Exemplo:***
  - ***Produto*** nº **1** (***Modern Science*** – o ***string Science*** é ***substituído*** por ***Physics***)
  - ***Não altera*** a ***linha real*** no **BD**, ***somente*** a ***linha retornada*** pela ***função*** é ***modificada***

```
SELECT REPLACE(nm_produto, 'Science', 'Physics')
FROM tb_produtos
WHERE id_produto = 1;
```

REPLACE(NM_PRODUTO,'SCIENCE','PHYSICS')
Modern Physics

Funções de Caracteres

- **SOUNDEX(x):**
  - ***Obtém*** um ***string contendo*** a ***representação fonética*** de "x"
  - ***Permite comparação*** de ***palavras homófonas***, mas ***não homógrafas*** (em ***inglês***)
- ***Exemplo:***
  - ***Recupera*** a ***coluna*** **SOBRENOME**, onde o ***sobrenome*** tem o ***som*** de "whyte"

```
SELECT sobrenome
FROM tb_clientes
WHERE SOUNDEX(sobrenome) = SOUNDEX('whyte');
```

SOBRENOME
White

Funções de Caracteres

- **SUBSTR**(x, início, [comprimento]):
  - ***Retorna*** um ***substring*** de "x" que ***inicia*** na ***posição determinada*** por início
  - ***Permite fornecer*** um comprimento (***opcional***) para o ***substring***
- ***Exemplo:***
  - ***Obter*** o ***substring*** de ***7 caracteres*** a ***partir*** da ***2ª posição*** da ***coluna*** **NM_PRODUTO** da **TB_PRODUTOS**

```
SELECT nm_produto, SUBSTR(nm_produto, 2, 7)
FROM tb_produtos
WHERE id_produto < 4;
```

NM_PRODUTO	SUBSTR(NM_PRODUTO,2,7)
Modern Science	odern S
Chemistry	hemistr
Supernova	upernov

Funções de Caracteres

- ***Uso*** de ***expressão*** com ***função***
- É ***possível fornecer*** qualquer ***expressão válida*** que seja ***avaliada*** como um ***string***
- ***Exemplo:***
  - ***Uso*** da ***função*** **SUBSTR** para ***obter*** o ***substring*** "**DBA**" do ***string*** "Administrador de Banco de Dados – DBA"

```
SELECT SUBSTR('Administrador de Banco de Dados - DBA', 34, 4)
FROM dual;
```

SUBSTR('ADMINISTRADORDEBANCODEDADOS-DBA',34,4)
DBA

Combinando Funções

- ***Permite utilizar combinação*** de ***funções*** em uma ***instrução SQL***
- ***Exemplo:***
  - ***Combinar*** as ***funções*** **UPPER** e **SUBSTR**
  - ***Observe*** que a ***saída*** de **SUBSTR** é ***passada*** para **UPPER**

```
SELECT nm_produto, UPPER(SUBSTR(nm_produto, 2, 8))
FROM tb_produtos
WHERE id_produto < 4;
```

NM_PRODUTO	UPPER(SUBSTR(NM_PRODUTO,2,8))
Modern Science	ODERN SC
Chemistry	HEMISTRY
Supernova	UPERNOVA

Funções Numéricas

- **ABS(x):**
  - ***Obtém*** o ***valor absoluto*** de "x"
  - O ***valor absoluto*** de um ***número*** é esse ***número sem nenhum sinal*** (positivo/negativo)
- ***Exemplo:***
  - ***Obter*** o ***valor absoluto*** da ***subtração*** de 30.00 da ***coluna*** **PRECO** da **TB_PRODUTOS** para os ***três primeiros produtos***

```
SELECT id_produto, preco, preco - 30.00, ABS(preco - 30.00)
FROM tb_produtos
WHERE id_produto < 4;
```

ID_PRODUTO	PRECO	PRECO-30.00	ABS(PRECO-30.00)
1	19.95	-10.05	10.05
2	30	0	0
3	25.99	-4.01	4.01

Funções Numéricas

- **CEIL(x):**
  - ***Obtém*** o ***menor inteiro maior*** ou ***igual*** a "x"
- ***Exemplo:***
  - ***Uso*** do **CEIL** para ***obter*** os ***valores absolutos*** de **5,8** e **-5,2**
  - O ***teto*** de **5,8** é **6**, ***porque*** é o ***menor inteiro maior*** do que **5,8**
  - O ***teto*** de **-5,2** é **-5**, ***porque*** **-5,2** é ***negativo*** e o ***menor inteiro maior*** do que ***este*** é **-5**

```
SELECT CEIL(5.8), CEIL(-5.2)
FROM dual;
```

CEIL(5.8)	CEIL(-5.2)
6	-5

Funções Numéricas

- **FLOOR(x):**
  - ***Obtém*** o ***maior inteiro menor*** ou ***igual*** a "x"
- ***Exemplo:***
  - ***Uso*** do **FLOOR** para ***obter*** os ***valores absolutos*** de **5,8** e **-5,2**
  - O ***piso*** de **5,8** é **5**, ***porque*** **5** é ***maior inteiro menor*** do que **5,8**
  - O ***piso*** de **-5,2** é **-6**, ***porque*** **-5,2** é ***negativo*** e o ***maior inteiro menor*** do que ***este*** é **-6**

```
SELECT FLOOR(5.8), FLOOR(-5.2)
FROM dual;
```

FLOOR(5.8)	FLOOR(-5.2)
5	-6

Funções Numéricas

- **MOD**(x, y):
  - ***Obtém*** o ***resto*** da ***divisão*** de "x" ***por*** "y"
- ***Exemplo:***
  - ***Uso*** do **MOD** para ***obter*** o ***resto***, ***quando*** **8** for ***dividido*** por **3** e ***por*** **4**

```
SELECT MOD(8,3), MOD(8,4)
FROM dual;
```

MOD(8,3)	MOD(8,4)
2	0

Funções Numéricas

- **POWER(x, y):**
  - ***Obtém*** o ***resultado*** de "x" ***elevado*** à ***potência*** "y"
- ***Exemplo:***
  - ***Uso*** do **POWER** para ***obter 2 elevado*** às ***potências*** **1** e **3**

```
SELECT POWER(2,1), POWER(2,3)
FROM dual;
```

POWER(2,1)	POWER(2,3)
2	8

Funções Numéricas

- **ROUND**(x, [y]):
  - ***Obtém*** o ***resultado*** do ***arredondamento*** de "x" com "y" ***casas decimais*** (***opcional***)
  - Se "y" por ***omitido***, "x" será ***arredondado*** com ***zero casa decimal***
  - Se "y" por ***negativo***, "x" será ***arredondado*** à ***esquerda*** do ***ponto decimal***

Funções Numéricas

- ***Exemplo:***
  - ***Uso*** do **ROUND** para ***obter*** o ***resultado*** do ***arredondamento*** de **5,75** com **zero**, **1** e **-1** ***casa decimal***

```
SELECT ROUND(5.75), ROUND(5.75, 1), ROUND(5.75, -1)
FROM dual;
```

ROUND(5.75)	ROUND(5.75,1)	ROUND(5.75,-1)
6	5.8	10

Funções Numéricas

- **SIGN(x):**
  - ***Obtém*** o ***sinal*** de "x"
  - ***Retorna* -1** de "x" for ***negativo***
  - ***Retorna* 1** se "x" for ***positivo***
  - ***Retorna* 0** se "x" for ***zero***
- ***Exemplo:***
  - ***Obter*** o ***sinal*** de **-5**, **5** e **0**

```
SELECT SIGN(-5), SIGN(5), SIGN(0)
FROM dual;
```

SIGN(-5)	SIGN(5)	SIGN(0)
-1	1	0

Funções Numéricas

- **SQRT(x):**
  - ***Obtém*** a ***raiz quadrada*** de "x"
- ***Exemplo:***
  - ***Obtém*** a ***raiz quadrada*** de **25** e **5**

```
SELECT SQRT(25), SQRT(5)
FROM dual;
```

SQRT(25)	SQRT(5)
5	2.23606797749978969640917366873127623544

Funções Numéricas

- **TRUNC(x, [y]):**
  - ***Utilizado*** para ***obter*** o ***resultado*** do ***truncamento*** de "x" ***com*** "y" ***casas decimais*** (***opcional***)
  - ***Se*** "y" for ***omitido***, "x" será ***truncado*** com ***zero casa decimal***
  - ***Se*** "y" for ***negativo***, "x" será ***truncado*** à ***esquerda*** do ***ponto decimal***
- ***Exemplo:***
  - ***Realiza*** o ***truncamento*** de **5,75** com ***zero***, **1** e **-1** ***casas decimais***

```
SELECT TRUNC(5.75), TRUNC(5.75, 1), TRUNC(5.75, -1)
FROM dual;
```

TRUNC(5.75)	TRUNC(5.75,1)	TRUNC(5.75,-1)
5	5.7	0

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Converte*** "x" em ***um string***
  - ***Permite fornecer*** o ***formato***, ***indicando*** o ***formato*** de "x" (***opcional***)
  - A ***estrutura*** do ***formato depende*** de "x" ser ***um número*** ou ***uma data***
- ***Exemplo (01):***
  - ***Realiza*** a ***conversão*** de um ***número*** em um ***string***

```
SELECT TO_CHAR(12345.67)
FROM dual;
```

TO_CHAR(12345.67)
12345.67

Funções de Conversão

- ***Exemplo (02):***
  - ***Realiza*** a ***conversão*** de um ***número*** em um ***string*** usando o ***formato*** 99,999.99

```
SELECT TO_CHAR(12345.67, '99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'99,999.99')
12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro 9: retorna dígitos*** na ***posição específica***, com um ***sinal negativo*** à ***esquerda*** se o ***número*** for ***negativo***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '99999.99')
FROM dual;
```

TO_CHAR(12345.67,'99999.99')
12345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro 0 (zero): retorna*** um ***número*** com ***zeros*** à ***esquerda*** e ou à ***direita***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '099,999.99')
FROM dual;
```

TO_CHAR(12345.67,'099,999.99')
012,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro . (ponto): retorna*** um ***ponto decimal*** em uma ***determinada posição***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'99,999.99')
12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro , (vírgula): retorna*** uma ***vírgula*** em uma ***determinada posição***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'99,999.99')
12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro $ (cifrão): retorna*** um ***cifrão*** à ***esquerda***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '$99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'$99,999.99')
$12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro B:*** se a ***parte inteira*** de um ***número*** de ***ponto fixo*** for ***zero***, ***retorna espaços*** para os ***zeros***

- ***Exemplo:***

```
SELECT TO_CHAR(00.67, 'B9.99')
FROM dual;
```

TO_CHAR(00.67,'B9.99')
.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro C: retorna*** o ***símbolo*** de ***moeda*** em uma ***posição específica***
  - O ***símbolo*** é ***extraído*** do ***parâmetro*** do **BD NLS_ISO_CURRENCY *definido*** pelo **DBA**

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, 'C99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'C99,999.99')
USD12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro D: retorna*** o ***símbolo*** de ***vírgula decimal*** em uma ***posição específica***
  - O ***símbolo*** é ***proveniente*** do ***parâmetro*** do **BD NLS_NUMERIC_CHARACTER** (o ***padrão*** é um ***caractere*** de ***ponto-final***
- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '99999D99')
FROM dual;
```

TO_CHAR(12345.67,'99999D99')
12345.67

Funções de Conversão

- **TO_CHAR(x, [format]):**
  - ***Parâmetro EEEE: retorna o número usando a notação científica***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '99999.99EEEE')
FROM dual;
```

TO_CHAR(12345.67,'99999.99EEEE')
1.23E+04

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro FM: remove*** os ***espaços*** à ***esquerda*** e à ***direita*** do ***número***

- ***Exemplo:***

```
SELECT TO_CHAR(0012345.6700, 'FM99999.99')
FROM dual;
```

TO_CHAR(0012345.6700,'FM99999.99')
12345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro G: retorna*** o ***símbolo*** de ***separador*** de ***grupo*** em uma ***posição específica***
  - O ***símbolo*** é ***proveniente*** do ***parâmetro*** do **BD NLS_NUMERIC_CHARACTER**

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '999G99')
FROM dual;
```

TO_CHAR(12345.67,'999G99')
123,46

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro L: retorna*** o ***símbolo*** de ***moeda local*** em uma ***posição específica***
  - O ***símbolo*** é ***oriundo*** do ***parâmetro*** do **BD NLS_CURRENCY**

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, 'L99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'L99,999.99')
$12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro MI: retorna*** um ***número negativo*** com um ***sinal*** de ***menos*** à ***direita***
  - ***Retorna*** um ***número positivo*** com um ***espaço*** à ***direita***

- ***Exemplo:***

```
SELECT TO_CHAR(-12345.67, '99,999.99MI')
FROM dual;
```

TO_CHAR(-12345.67,'99,999.99MI')
12,345.67-

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro PR: retorna*** um ***número negativo*** entre os ***sinais*** de ***menor*** e ***maior*** (**< >**)
  - ***Retorna*** um ***número positivo*** com um ***espaço*** à ***esquerda*** e à ***direita***

- ***Exemplo:***

```
SELECT TO_CHAR(-12345.67, '99,999.99PR')
FROM dual;
```

TO_CHAR(-12345.67,'99,999.99PR')
<12,345.67>

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro RN: retorna*** o ***número*** como ***algarismos romanos***
  - **RN** ***retorna números maiúsculo***; **rn** ***retorna números minúsculos***
  - O ***número*** deve ser um ***valor inteiro*** entre **1** e **3999**

- ***Exemplo:***

```
SELECT TO_CHAR(2013, 'RN')
FROM dual;
```

TO_CHAR(2013,'RN')
MMXIII

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro TM: retorna*** o ***número*** usando a ***quantidade mínima*** de ***caracteres***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, 'TM')
FROM dual;
```

TO_CHAR(12345.67,'TM')
12345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro S: retorna*** um ***número negativo*** com um ***sinal*** de ***negativo*** à ***esquerda***; ***retorna*** um ***número positivo*** com um ***sinal*** de ***positivo*** à ***esquerda***
  - ***Parâmetro 999S: retorna*** um ***número negativo*** com um ***sinal*** de ***negativo*** à ***direita***; ***retorna*** um ***número positivo*** com um ***sinal*** de ***positivo*** à ***direita***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, 'S99999.99')
FROM dual;
```

TO_CHAR(12345.67,'S99999.99')
+12345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro U:*** ***retorna*** o ***símbolo*** de ***moeda*** (**Euro**, por ***exemplo***) em uma ***posição específica***
  - O ***símbolo*** é ***oriundo*** do ***parâmetro*** do **BD NLS_DUAL_CORRENCY**

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, 'U99,999.99')
FROM dual;
```

TO_CHAR(12345.67,'U99,999.99')
$12,345.67

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Parâmetro V: retorna*** o ***número multiplicado*** por ***10x***, onde "x" é o ***número*** de ***caracteres 9 após*** a ***letra*** "**V**"
  - Se for ***necessário***, o ***número*** é ***arredondado***

- ***Exemplo:***

```
SELECT TO_CHAR(12345.67, '99999V99')
FROM dual;
```

TO_CHAR(12345.67,'99999V99')
1234567

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Retornará*** um ***string*** de ***caracteres*** "***#***" se, eventualmente, for ***formatado*** um ***número contendo dígitos*** a ***mais*** para o ***formato***

- ***Exemplo:***

```
SELECT TO_CHAR(12345678.90, '99,999.99')
FROM dual;
```

TO_CHAR(12345678.90,'99,999.99')
##########

Funções de Conversão

- **TO_CHAR**(x, [format]):
  - ***Uso*** do **TO_CHAR** para ***converter colunas*** contendo ***números*** em ***strings***
- ***Exemplo:***
  - ***Converte*** a ***coluna*** **PRECO** da **TB_PRODUTOS** em um ***string***

```
SELECT id_produto, 'O preço do produto é: ' || TO_CHAR(preco, 'L99.99')
FROM tb_produtos
WHERE id_produto < 5;
```

ID_PRODUTO	'OPREÇODOPRODUTOÉ:'||TO_CHAR(PRECO,'L99.99')
1	0 preço do produto é: $19.95
2	0 preço do produto é: $30.00
3	0 preço do produto é: $25.99
4	0 preço do produto é: $13.95

Funções de Conversão

- **TO_NUMBER**(x, [format]):
  - ***Converte*** "**x**" em um ***número***
  - ***Permite fornecer*** um ***string*** de ***formato*** para ***indicar*** o ***formato*** de "**x**" (***opcional***)
- ***Exemplo (01):***
  - ***Converte*** o ***string*** *970.13* em um ***número***

```
SELECT TO_NUMBER('970.13')
FROM dual;
```

TO_NUMBER('970.13')
970.13

Funções de Conversão

- **TO_NUMBER**(x, [format]):
  - ***Converte*** "**x**" em um ***número***
  - ***Permite fornecer*** um ***string*** de ***formato*** para ***indicar*** o ***formato*** de "**x**" (***opcional***)
- ***Exemplo (02):***
  - ***Converte*** o ***string*** *970.13* em um ***número***, posteriormente, ***soma*** *25.50* a esse ***número***

```
SELECT TO_NUMBER('970.13') + 25.50
FROM dual;
```

TO_NUMBER('970.13')+25.50
995.63

Funções de Conversão

- **TO_NUMBER**(x, [format]):
  - ***Converte*** "**x**" em um ***número***
  - ***Permite fornecer*** um ***string*** de ***formato*** para ***indicar*** o ***formato*** de "**x**" (***opcional***)
- ***Exemplo (03):***
  - ***Converte*** o ***string*** **-$**12,345.67 em um ***número***, ***passando*** o ***string*** de ***formato*** **$**99,999.99 para **TO_NUMBER**

```
SELECT TO_NUMBER('-$12,345.67', '$99,999.99')
FROM dual;
```

TO_NUMBER('-$12,345.67','$99,999.99')
-12345.67

Funções de Conversão

- **CAST**(x AS tipo):
  - ***Converte*** "x" em um ***tipo*** de ***dado compatível***, ***especificado*** por "tipo"
- ***Exemplo (01):***
  - ***Converte valores literais*** em ***tipos específicos***

```
SELECT
  CAST(12345.67 AS VARCHAR2(10)),
  CAST('9A4F' AS RAW(2)),
  CAST('01-DEC-2007' AS DATE),
  CAST(12345.678 AS NUMBER(10,2))
FROM dual;
```

CAST(12345.67ASVARCHAR2(10))	CAST('9A4F'ASRAW(2))	CAST('01-DEC-2007'ASDATE)	CAST(12345.678ASNUMBER(10,2))
12345.67	9A4F	01-DEC-07	12345.68

Funções de Conversão

- **CAST**(x AS tipo):
  - ***Converte*** "x" em um ***tipo*** de ***dado compatível***, ***especificado*** por "tipo"
- ***Exemplo (02):***
  - ***Converte valores*** da ***coluna***, de um ***tipo*** para ***outro***

```
SELECT
  CAST(preco AS VARCHAR2(10)),
  CAST(preco + 2 AS NUMBER(7,2)),
  CAST(preco AS BINARY_DOUBLE)
FROM tb_produtos
WHERE id_produto = 1;
```

CAST(PRECOASVARCHAR2(10))	CAST(PRECO+2ASNUMBER(7,2))	CAST(PRECOASBINARY_DOUBLE)
19.95	21.95	19.95

Funções de Expressão Regular

- ***Permitem procurar*** um ***padrão*** de ***caracteres*** em um ***string***
- ***Exemplo:***
  - ***Imagine*** uma ***lista*** de ***anos*** (**1965**, **1968**, **1971** e **1970**)
  - ***Obter*** os ***anos*** de **1965** a **1968**
  - ***Expressão regular utilizada:***

    ^196[5 – 8]$

Funções de Expressão Regular

- A ***expressão regular*** contém ***alguns metacaracteres***
- ***Exemplo:*** **^**196[5 – 8]**$**
  - **^: *corresponde*** à ***posição inicial*** do ***string***
  - [**5 – 8**]: ***corresponde*** aos ***caracteres*** entre **5** e **8**
  - **$: *corresponde*** à ***posição final*** de um ***string***
- ***Portanto:***
  - **^**196: ***corresponde*** a um ***string*** que ***inicia*** em **196**
  - [**5-8**]**$: *corresponde*** a um ***string*** que ***termina*** com **5**, **6**, **7** ou **8**
  - Ou seja, a ***expressão*** **^**196[5-8]**$ *corresponde*** a **1965**, **1966**, **1967** e **1968**

Funções de Expressão Regular

- **REGEXP_LIKE**(x, padrão, [opção_correspondência])
  - ***Procura*** "x" a ***expressão regular definida*** no ***parâmetro padrão***
  - ***Permite*** também ***fornecer*** uma "opção_correspondência" (***opcional***), a qual pode ser ***configurada*** com um dos ***seguintes caracteres:***
    - **c**: ***determina*** a ***correspondência com diferenciação*** de ***maiúsculo*** e ***minúsculo*** (***padrão***)
    - **i**: ***determina*** a ***correspondência sem diferenciação*** de ***maiúsculo*** e ***minúsculo***
    - **n**: ***permite*** usar o ***operador*** de ***correspondência*** com ***qualquer caractere***

Funções de Expressão Regular

- **REGEXP_LIKE**(x, padrão, [opção_correspondência])
  - ***Procura*** "x" a ***expressão regular definida*** no ***parâmetro padrão***
  - ***Permite*** também ***fornecer*** uma "opção_correspondência" (***opcional***), a qual pode ser ***configurada*** com um dos ***seguintes caracteres*** *(continuação)*:
    - **m**: ***trata*** "x" como uma ***linha múltipla***

Funções de Expressão Regular

- **REGEXP_LIKE**(x, padrão, [opção_correspondência])
- ***Exemplo (01):***
  - ***Recupera*** os ***clientes*** cuja ***data*** de ***nascimento*** esteja ***entre*** **1965** e **1968**

```
SELECT id_cliente, nome, sobrenome, dt_nascimento
FROM tb_clientes
WHERE REGEXP_LIKE(TO_CHAR(dt_nascimento, 'YYYY'), '196[5-8]$');
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO
1	John	Brown	01-JAN-65
2	Cynthia	Green	05-FEB-68

Funções de Expressão Regular

- **REGEXP_LIKE**(x, padrão, [opção_correspondência])
- ***Exemplo (02):***
  - ***Recupera*** os ***clientes*** cujo ***nome inicia*** com **J** ou **j**
  - **i:** ***indica correspondência sem diferenciação*** de ***maiúsculo*** e ***minúsculo***

```
SELECT id_cliente, nome, sobrenome, dt_nascimento
FROM tb_clientes
WHERE REGEXP_LIKE(nome, 'j', 'i');
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO
1	John	Brown	01-JAN-65

Funções de Expressão Regular

- **REGEXP_INSTR**(x, padrão, [início], [ocorrência], [opção_de_retorno], [opção_de_correspondência])
  - ***Procura*** o ***padrão*** em "x"
- ***Exemplo:***
  - ***Retornar*** a ***posição correspondente*** à ***expressão regular***

```
SELECT
  REGEXP_INSTR('Teste  de expressão regular', 'e', 6, 2) AS resultado
FROM dual;
```

RESULTADO
10

Funções de Expressão Regular

- **REGEXP_REPLACE**(x, padrão, [string_substituto], [início], [ocorrência], [opção_de_correspondência])
  - ***Procura*** o ***padrão*** em "x" e ***substitui*** pelo string_substituto
- ***Exemplo:***
  - ***Substitui*** o ***substring correspondente*** à ***expressão regular***

```
SELECT
  REGEXP_REPLACE('Teste  de expressão regular', 'd[[:alpha:]]{1}', 'Oracle')
AS resultado
FROM dual;
```

RESULTADO
Teste Oracle expressão regular

Funções de Expressão Regular

- **REGEXP_SUBSTR**(x, padrão, [início], [ocorrência], [opção_de_correspondência])
  - ***Obtém*** um ***string*** de "x" ***correspondente*** ao ***padrão***
  - A ***busca inicia-se*** na ***posição determinada*** por "início"
- ***Exemplo:***
  - ***Retorna*** o ***substring correspondente*** à ***expressão regular***

```
SELECT
  REGEXP_SUBSTR('Teste de expressão regular', 'e[[:alpha:]]{8}') AS resultado
FROM dual;
```

RESULTADO
expressão

Funções de Expressão Regular

- **REGEXP_COUNT**(x, padrão, [início], [opção_correspondência])
  - ***Procura*** "padrão" em "x" e ***obtém*** o ***número*** de ***vezes*** que o "padrão" é ***encontrado*** em "x"
- ***Exemplo:***
  - ***Retorna*** o ***número*** de ***vezes*** que a ***expressão regular ocorre*** em um ***string***

```
SELECT
  REGEXP_COUNT('teste teste teste expressão regular', 't[[:alpha:]]{4}') AS resultado
FROM dual;
```

RESULTADO
3

Funções Agregadas

Funções Agregadas

- ***Operam*** em ***grupo*** de ***linhas*** (***tuplas***) e ***retornam*** uma ***linha*** de ***saída***
- As ***funções agregadas*** também são ***conhecidas*** como ***funções*** de ***grupo***, pois ***operam*** em um ***grupo*** de ***linhas***
- ***Observação:***
  - ***Permite utilizar funções agregadas*** com ***qualquer expressão válida***
  - ***Exemplo:*** **COUNT**, **MAX** e **MIN** com ***números***, ***strings*** e ***data/hora***

Funções Agregadas

- ***Observação*** (*continuação*)***:***
  - ***Valores nulos*** são ***ignorados*** pelas ***funções agregadas*** (***nulo*** indica um ***valor desconhecido***)
  - ***Uso*** da ***palavra-chave*** **DISTINCT** em uma ***função agregada*** para ***excluir entradas duplicadas***

Funções Agregadas

- **AVG**(x)
  - ***Obtém*** o ***valor médio*** de "x"
- ***Exemplo (01):***
  - ***Obter*** o ***preço médio*** dos ***produtos***

```
SELECT AVG(preco)
FROM tb_produtos;
```

AVG(PRECO)
19.730833333333333333333333333333333333333

Funções Agregadas

- **AVG**(x)
  - ***Obtém*** o ***valor médio*** de "x"
- ***Exemplo (02):***
  - ***Adiciona*** 2.00 ao ***preço*** de ***cada linha*** e, posteriormente, ***retorna*** a ***média*** desses ***valores***

```
SELECT AVG(preco + 2.00)
FROM tb_produtos;
```

AVG(PRECO+2.00)
21.730833333333333333333333333333333333333

Funções Agregadas

- **AVG**(x)
  - ***Obtém*** o ***valor médio*** de "x"
- ***Exemplo (03):***
  - ***Uso*** da ***palavra-chave*** **DISTINCT** para ***excluir valores idênticos*** na ***coluna*** **PRECO** ao ***calcular*** a ***média*** usando **AVG**

```
SELECT AVG(DISTINCT preco)
FROM tb_produtos;
```

AVG(DISTINCTPRECO)
20.298181818181818181818181818181818182

Funções Agregadas

- **COUNT**(x)
  - ***Obtém*** o ***número*** de ***linhas retornadas*** por uma ***consulta***
- ***Exemplo (01):***
  - ***Obter*** o ***número*** de ***linhas*** na ***tabela*** **TB_PRODUTOS** usando **COUNT**

```
SELECT COUNT(id_produto)
FROM tb_produtos;
```

COUNT(ID_PRODUTO)
12

Funções Agregadas

- **COUNT(x)**
  - ***Evitar*** o ***uso*** de ***asterisco*** com a ***função*** **COUNT**
  - ***Asterisco*** pode fazer **COUNT** ***demorar*** mais para ***retornar*** o ***resultado***
  - ***Deve-se utilizar*** uma ***coluna*** da ***tabela*** ou a ***pseudocoluna*** **ROWID**
- ***Exemplo (02):***
  - ***Informa*** **ROWID** para **COUNT** e ***obtém*** o ***número*** de ***linhas*** da **TB_PRODUTOS**

```
SELECT COUNT(ROWID)
FROM tb_produtos;
```

COUNT(ROWID)
12

Funções Agregadas

- **MAX**(x) e **MIN**(x)
  - ***Obtém*** os ***valores máximo*** e ***mínimo*** de "x"
- ***Exemplo (01):***
  - ***Obter*** os ***valores máximo*** e ***mínimo*** da ***coluna*** **PRECO** da **TB_PRODUTOS** usando **MAX** e **MIN**

```
SELECT MAX(preco), MIN(preco)
FROM tb_produtos;
```

MAX(PRECO)	MIN(PRECO)
49.99	10.99

Funções Agregadas

- **MAX**(x) e **MIN**(x)
  - ***Permite utilizar*** **MAX** e **MIN** com ***qualquer tipo*** de ***dado***, ***inclusive strings*** e ***datas***
  - **MAX** com ***string***, são ***classificados*** em ***ordem alfabética***, com o ***string*** "***máximo***" no ***final*** da ***lista*** e o ***string*** "***mínimo***" no ***início***
- ***Exemplo (02):***
  - ***Obter*** os ***strings*** ***máximo*** e ***mínimo*** da ***coluna*** **NM_PRODUTO** da **TB_PRODUTOS**

```
SELECT MAX(nm_produto), MIN(nm_produto)
FROM tb_produtos;
```

MAX(NM_PRODUTO)	MIN(NM_PRODUTO)
Z Files	2412: The Return

Funções Agregadas

- **MAX**(x) e **MIN**(x)
  - A ***data*** "***máxima***" ***ocorre*** no ***ponto*** mais ***recente*** no ***tempo*** e a ***data*** "***mínima***", no ***ponto*** mais ***antigo***
- ***Exemplo (03):***
  - ***Obter*** o ***valor máximo*** e ***mínimo*** de **DT_NASCIMENTO** da **TB_CLIENTES**

```
SELECT MAX(dt_nascimento), MIN(dt_nascimento)
FROM tb_clientes;
```

MAX(DT_NASCIMENTO)	MIN(DT_NASCIMENTO)
16-MAR-71	01-JAN-65

Funções Agregadas

- **STDDEV(x)**
  - ***Obtém*** o ***desvio padrão*** de "x"
  - O ***desvio padrão*** é uma ***função estatística***, e é ***definido*** como a ***raiz quadrada*** da ***variância***
- ***Exemplo:***
  - ***Obter*** o ***desvio padrão*** dos ***valores*** da ***coluna*** **PRECO** da **TB_PRODUTOS**

```
SELECT STDDEV(preco)
FROM tb_produtos;
```

Funções Agregadas

- **SUM**(x)
  - ***Soma todos*** os ***valores presentes*** em "x", ***retornando*** o ***total***
- ***Exemplo:***
  - ***Obter*** a ***soma*** da ***coluna*** **PRECO** da **TB_PRODUTOS**

```
SELECT SUM(preco)
FROM tb_produtos;
```

SUM(PRECO)
236.77

Funções Agregadas

- **VARIANCE(x)**
  - ***Obtém*** a ***variância*** de "x"
  - A ***variância*** é uma ***função estatística definida*** como a ***dispersão*** ou ***variação*** de um ***grupo*** de ***números*** em uma ***amostra***
  - É ***equivalente*** ao ***quadrado*** do ***desvio padrão***
- ***Exemplo:***
  - ***Obter*** a ***variância*** dos ***valores*** da ***coluna*** **PRECO** da **TB_PRODUTOS**

```
SELECT VARIANCE(preco)
FROM tb_produtos;
```

VARIANCE(PRECO)
122.979899242424242424242424242424242424

Agrupando Linhas

Agrupando Linhas

- ***Permite agrupar linhas*** em uma ***tabela*** e ***obter*** alguma ***informação*** sobre esses ***grupos*** de ***linhas***
- ***Exemplo:***
  - ***Obter*** o ***preço médio*** dos ***diferentes tipos*** de ***produtos*** da **TB_PRODUTOS**
  - ***Facilitador:***
    - ***Utilização*** do **GROUP BY** para ***agrupar*** as ***linhas semelhantes***
    - A ***cláusula*** **GROUP BY** é ***utilizada*** para ***agrupar linhas*** em ***blocos*** com um ***valor comum*** de ***coluna***

Agrupando Linhas

- ***Exemplo:***
  - ***Agrupar*** as ***linhas*** da **TB_PRODUTOS** em ***blocos*** com o ***mesmo valor*** de **ID_TIPO_PRODUTO**

```
SELECT id_tipo_produto
FROM tb_produtos
GROUP BY id_tipo_produto;
```

ID_TIPO_PRODUTO
(null)
1
2
4
3

Agrupando Linhas

- **GROUP BY**
  - ***Usando várias colunas*** em um ***grupo***, ou seja, ***permite especificar várias colunas*** em uma ***cláusula*** **GROUP BY**
- ***Exemplo:***
  - ***Inclusão*** das ***colunas*** **ID_PRODUTO** e **ID_CLIENTE** da **TB_COMPRAS** em uma **cláusula GROUP BY**

```
SELECT id_produto, id_cliente
FROM tb_compras
GROUP BY id_produto, id_cliente;
```

ID_PRODUTO	ID_CLIENTE
1	1
1	2
2	2
3	1
4	1

Agrupando Linhas

- **GROUP BY**
  - ***Usando grupos*** de ***linhas*** com ***função agregada***, permitindo ***efetuar*** o ***cálculo*** no ***grupo*** de ***linhas*** em ***cada bloco***, ***retornando*** um ***valor*** por ***bloco***
- ***Exemplo (01):***
  - ***Obter*** o ***número*** de ***linhas*** com o ***mesmo valor*** de **ID_TIPO_PRODUTO** da **TB_PRODUTOS**

```
SELECT id_tipo_produto, COUNT(ROWID)
FROM tb_produtos
GROUP BY id_tipo_produto
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	COUNT(ROWID)
1	2
2	4
3	2
4	3
(null)	1

Agrupando Linhas

- **GROUP BY**
- ***Exemplo (02):***
  - ***Obter*** o ***preço médio*** dos ***diferentes tipos*** de ***produtos*** da **TB_PRODUTOS**

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
GROUP BY id_tipo_produto
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	AVG(PRECO)
1	24.975
2	26.22
3	13.24
4	13.99
(null)	13.49

Agrupando Linhas

- **GROUP BY**
- ***Exemplo (03):***
  - ***Obter*** a ***variância*** do ***preço*** dos ***produtos*** para cada ***valor*** de **ID_TIPO_PRODUTO**

```
SELECT id_tipo_produto, VARIANCE(preco)
FROM tb_produtos
GROUP BY id_tipo_produto
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	VARIANCE(PRECO)
1	50.50125
2	280.8772
3	0.125
4	7
(null)	0

Agrupando Linhas

- **GROUP BY**
- ***Exemplo (04):***
  - ***Inclusão*** de uma ***chamada*** de ***função agregada*** na ***cláusula*** **ORDER BY**

```
SELECT VARIANCE(preco)
FROM tb_produtos
GROUP BY id_tipo_produto
ORDER BY VARIANCE(preco);
```

VARIANCE(PRECO)
0
0.125
7
50.50125
280.8772

Agrupando Linhas

- ***Utilização incorreta*** de ***chamadas*** de ***funções agregadas***
  - ***Quando*** sua ***consulta contém*** uma ***função agregada***, e ***recupera colunas não inseridas dentro*** de uma ***função agregada***
  - Essas ***colunas devem*** ser ***inseridas*** em uma ***cláusula*** **GROUP BY**

Agrupando Linhas

- ***Exemplo:***
  - ***Tentar recuperar*** a ***coluna*** **ID_TIPO_PRODUTO** e **AVG(*preco*)**, mas ***omitir*** uma ***cláusula*** **GROUP BY** para o **ID_TIPO_PRODUTO**

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos;
```

```
ORA-00937: not a single-group group function
00937. 00000 - "not a single-group group function"
*Cause:
*Action:
Error at Line: 1 Column: 7
```

Agrupando Linhas

- ***Observação:***
  - ***Quando*** uma ***consulta contém*** uma ***função agregada*** e ***recupera colunas não inseridas dentro*** de uma ***função agregada***, essas ***colunas devem*** ser ***inseridas*** em uma ***cláusula*** **GROUP BY**
  - ***Não*** é ***permitido*** também ***utilizar função agregada*** para ***limitar*** uma ***cláusula*** **WHERE**

Agrupando Linhas

- ***Exemplo:***

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
WHERE AVG(PRECO) > 20.00
GROUP BY id_tipo_produto;
```

```
ORA-00934: group function is not allowed here
00934. 00000 - "group function is not allowed here"
*Cause:
*Action:
Error at Line: 3 Column: 6
```

Agrupando Linhas

- ***Usando*** a ***cláusula*** **HAVING** para ***filtrar grupos*** de ***linhas***
- **HAVING:**
  - ***Usado*** para ***filtrar grupos*** de ***linhas***, ***inserida*** posteriormente à ***cláusula*** **GROUP BY**
- ***Observação:***
  - **GROUP BY** pode ser ***usada sem*** **HAVING**, mas **HAVING** ***deve*** ser ***usada*** em ***conjunto*** com **GROUP BY**

Agrupando Linhas

- ***Exemplo:***
  - ***Obter*** os ***tipos*** de ***produtos*** que têm ***preços médio maior*** que **R$ 20,00**
  - **GROUP BY** para ***agrupar*** as ***linhas*** em ***blocos*** com o ***mesmo valor*** de **ID_TIPO_PRODUTO**
  - **HAVING** para ***limitar*** os ***resultados retornados*** aos ***grupos*** que ***têm*** um ***preço médio maior*** do que **R$ 20,00**

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
GROUP BY id_tipo_produto
HAVING AVG(preco) > 20.00;
```

ID_TIPO_PRODUTO	AVG(PRECO)
1	24.975
2	26.22

Agrupando Linhas

- ***Usando*** as ***cláusulas*** **WHERE** e **GROUP BY** ***juntas***
  - ***Primeiro*** a ***cláusula*** **WHERE** ***filtra*** as ***linhas retornadas***, posteriormente, a ***cláusula*** **GROUP BY** ***agrupa*** as ***linhas restantes*** em ***blocos***

Agrupando Linhas

- ***Exemplo:***
  - **WHERE *filtra*** as ***linhas*** da **TB_PRODUTOS**, ***selecionando*** aquelas cujo ***valor*** de **PRECO** é ***menor*** que **R$ 15.00**
  - **GROUP BY** para ***agrupar*** as ***linhas restantes*** pela ***coluna*** **ID_TIPO_PRODUTO**

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
WHERE preco < 15.00
GROUP BY id_tipo_produto
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	AVG(PRECO)
2	14.45
3	13.24
4	12.99
(null)	13.49

Agrupando Linhas

- ***Usando*** as ***cláusulas*** **WHERE**, **GROUP BY** e **HAVING** ***juntas***
  - ***Primeiro*** a ***cláusula*** **WHERE** ***filtra*** as ***linhas***, posteriormente, a ***cláusula*** **GROUP BY** ***agrupa*** as ***linhas restantes*** em ***blocos***, ***finalizando***, a ***cláusula*** **HAVING** ***filtra*** os ***grupos*** de ***linhas***

Agrupando Linhas

- ***Exemplo:***
  - **WHERE *filtra*** as ***linhas*** da **TB_PRODUTOS**, ***selecionando*** aquelas cujo ***valor*** de **PRECO** é ***menor*** que **R$ 15.00**
  - **GROUP BY** para ***agrupar*** as ***linhas restantes*** pela ***coluna*** **ID_TIPO_PRODUTO**
  - **HAVING *filtra*** os ***grupos*** de ***linhas***, ***selecionando*** aqueles cujo **PRECO *médio*** é ***maior*** do que **R$ 13.00**

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
WHERE preco < 15.00
GROUP BY id_tipo_produto
HAVING AVG(preco) > 13.00
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	AVG(PRECO)
2	14.45
3	13.24
(null)	13.49

Manipulando Data e Hora

Data e Hora

- ***Armazenando*** e ***Processamento data*** e ***hora***
- ***Data/hora*** é ***armazenado usando*** o ***tipo*** **DATE**
- **DATE** ***armazena*** o ***século***, ***todos*** os ***quatro dígitos*** de um ***ano***, o ***mês***, o ***dia***, a ***hora*** (no ***formato 24 horas***), o ***minuto*** e o ***segundo***
- **TIMESTAMP** ***armazena*** o ***século***, ***todos*** os ***quatro dígitos*** de um ***ano***, o ***mês***, o ***dia***, a ***hora*** (no ***formato 24 horas***), o ***minuto*** e o ***segundo***
  - ***Vantagens:*** pode ***armazenar frações*** de ***segundo*** e ***fuso horário***

Data e Hora

- ***Armazenamento*** e ***recuperação*** de ***datas***
- Por ***padrão***, o **SGBD *utiliza*** o ***formato*** **DD-MON-YYYY**, ***onde:***
  - **DD** é um ***dia*** com ***dois dígitos***
  - **MON** são as ***três primeiras letras*** do ***mês***
  - **YYYY** é ***um ano*** de ***quatro dígitos***

Data e Hora

- ***Armazenamento*** e ***recuperação*** de ***datas***
- ***Exemplo:***
  - ***Inserir*** uma ***linha*** na **TB_CLIENTES** que ***contém*** uma ***coluna*** do ***tipo*** **DATE** ***nomeada*** de **DT_NASCIMENTO**
  - ***Palavra-chave*** **DATE** ***fornece*** uma ***data literal*** ao **BD**
  - A ***data*** deve usar o ***formato*** de ***data*** do ***padrão*** **ANSI YYYY-MM-DD**

```
INSERT INTO tb_clientes (id_cliente, nome, sobrenome, dt_nascimento,
                         telefone, fg_ativo)
VALUES (7, 'Steve', 'Purple', DATE '1972-10-25', '800-555-1215', 1);
```

Data e Hora

- ***Observação:***
  - ***Usar datas*** no ***padrão*** **ANSI** em ***instruções*** **SQL** apresenta ***vantagem***, ***permitindo*** que essas ***instruções possam*** ser ***executadas*** em **BD** que ***não*** sejam o ***Oracle***
- ***Exemplo:***
  - ***Recuperar*** as ***linhas*** da **TB_CLIENTES** e, posteriormente, ***executar*** a ***instrução*** de **ROLLBACK**

```
SELECT *
FROM tb_clientes
ORDER BY id_cliente;

ROLLBACK;
```

Convertendo Data/Hora

- ***Uso*** das ***funções*** **TO_CHAR** e **TO_DATE** para ***converter*** uma ***data/hora*** em um ***string*** e ***vice-versa***
- **TO_CHAR**(x, [string]): ***converte*** a ***data/hora*** em um ***string***
- ***Exemplo (01):***
  - ***Converte*** a ***coluna*** **DT_NASCIMENTO** da **TB_CLIENTES** em um ***string*** com o ***formato*** **MONTH DD, YYYY**

```
SELECT id_cliente, TO_CHAR(dt_nascimento, 'MONTH DD, YYYY')
FROM tb_clientes;
```

Convertendo Data/Hora

- **TO_CHAR**(x, [string]): ***converte*** a ***data/hora*** em um ***string***
- ***Exemplo (02):***
  - ***Obtém*** a ***data*** e ***hora atual*** do **BD** ***usando*** a ***função*** **SYSDATE**, posteriormente, ***converte*** a ***data*** e ***hora*** em um ***string*** usando **TO_CHAR** com ***formato*** de ***24 horas***

```
SELECT TO_CHAR(SYSDATE, 'MONTH DD, YYYY, HH24:MI:SS')
FROM dual;
```

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Century	CC	Two-digit century.	21
	SCC	Two-digit century with a negative sign (–) for B.C.	–10
Quarter	Q	One-digit quarter of the year.	1
Year	YYYY	All four digits of the year.	2008
	IYYY	All four digits of the ISO year.	2008
	RRRR	All four digits of the rounded year (governed by the present year). See the section "How Oracle Interprets Two-Digit Years"	2008

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Year	SYYYY	All four digits of the year with a negative sign (–) for B.C.	–1001
	Y,YYY	All four digits of the year, with a comma after the first digit.	2,008
	YYY	Last three digits of the year.	008
	IYY	Last three digits of the ISO year.	008
	YY	Last two digits of the year.	08
	IY	Last two digits of the ISO year.	06
	RR	Last two digits of the rounded year, which depend on the present year. See the section "How Oracle Interprets Two-Digit Years" later in this chapter for details.	08
	Y	Last digit of the year.	8
	I	Last digit of the ISO year.	8
	YEAR	Name of the year in uppercase.	TWO THOUSAND-EIGHT
	Year	Name of the year with the first letter in uppercase.	Two Thousand-Eight

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Month	MM	Two-digit month of the year.	01
	MONTH	Full name of the month in uppercase, right-padded with spaces to a total length of nine characters.	JANUARY
	Month	Full name of the month with first letter in uppercase, right-padded with spaces to a total length of nine characters.	January
	MON	First three letters of the name of the month in uppercase.	JAN
	Mon	First three letters of the name of the month with the first letter in uppercase.	Jan
	RM	Roman numeral month.	The Roman numeral month for the fourth month (April) is IV.

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Week	WW	Two-digit week of the year.	02
	IW	Two-digit ISO week of the year.	02
	W	One-digit week of the month.	2
Day	DDD	Three-digit day of the year.	103
	DD	Two-digit day of the month.	31
	D	One-digit day of the week.	5
	DAY	Full name of the day in uppercase.	SATURDAY
	Day	Full name of the day with the first letter in uppercase.	Saturday
	DY	First three letters of the name of the day in uppercase.	SAT
	Dy	First three letters of the name of the day with the first letter in uppercase.	Sat
	J	Julian day—the number of days that have passed since January 1, 4713 B.C.	2439892

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Hour	HH24	Two-digit hour in 24-hour format.	23
	HH	Two-digit hour in 12-hour format.	11
Minute	MI	Two-digit minute.	57
Second	SS	Two-digit second.	45
	FF[1..9]	Fractional seconds with an optional number of digits to the right of the decimal point. Only applies timestamps, which you'll learn about in the section "Using Timestamps" later in this chapter.	When dealing with 0.123456789 seconds, FF3 would round the seconds to 0.123.
	SSSSS	Number of seconds past 12 A.M.	46748
	MS	Millisecond (millionths of a second).	100
	CS	Centisecond (hundredths of a second).	10

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Separators	`-/,.:: "text"`	Characters that allow you to separate the aspects of a date and time. You can supply freeform text in quotes as a separator.	For the date December 13, 1969, `DD-MM-YYYY` would produce `12-13-1969`, and `DD/MM/YYYY` would produce `12/13/1969`.
Suffixes	`AM` or `PM`	AM or PM as appropriate.	`AM`
	`A.M.` or `P.M.`	A.M. or P.M. as appropriate.	`P.M.`
	`AD` or `BC`	AD or BC as appropriate.	`AD`

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Suffixes	A.D. or B.C.	A.D. or B.C. as appropriate.	B.C.
	TH	Suffix to a number. You can make the suffix uppercase by specifying the numeric format in uppercase and vice versa for lowercase.	For a day number of 28, ddTH would produce 28TH, and DDTH would produce 28TH.
	SP	Number is spelled out.	For a day number of 28, DDSP would produce TWENTY-EIGHT, and ddSP would produce twenty-eight.
	SPTH	Combination of TH and SP.	For a day number of 28, DDSPTH would produce TWENTY-EIGHTH, and ddSPTH would produce twenty-eighth.

Convertendo Data/Hora

- ***Parâmetros*** de ***formatação*** de ***data/hora***

Aspect	Parameter	Description	Example
Era	EE	Full era name for Japanese Imperial, ROC Official, and Thai Buddha calendars.	No example
	E	Abbreviated era name.	No example
Time zones	TZH	Time zone hour. You'll learn about time zones later in the section "Using Time Zones."	12
	TZM	Time zone minute.	30
	TZR	Time zone region.	PST
	TZD	Time zone with daylight savings information.	No example

Convertendo Data/Hora

- A ***tabela abaixo*** apresenta ***exemplos*** de ***strings*** para ***formatar*** a ***data*** **5** de **fevereiro** de **1968**, juntamente com o ***string retornado*** de ***uma chamada*** de **TO_CHAR**

Format String	Returned String
MONTH DD, YYYY	FEBRUARY 05, 1968
MM/DD/YYYY	02/05/1968
MM-DD-YYYY	02-05-1968
DD/MM/YYYY	05/02/1968
DAY MON, YY AD	MONDAY FEB, 68 AD
DDSPTH "of" MONTH, YEAR A.D.	FIFTH of FEBRUARY, NINETEEN SIXTY-EIGHT A.D.
CC, SCC	20, 20
Q	1

Convertendo Data/Hora

- A ***tabela abaixo*** apresenta ***exemplos*** de ***strings*** para ***formatar*** a ***data*** **5** de **fevereiro** de **1968**, juntamente com o ***string retornado*** de ***uma chamada*** de **TO_CHAR**

Format String	Returned String
YYYY, IYYY, RRRR, SYYYY, Y,YYY, YYY, IYY, YY, IY, RR, Y, I, YEAR, Year	1968, 1968, 1968, 1968, 1,968, 968, 968, 68, 68, 68, 8, 8, NINETEEN SIXTY-EIGHT, Nineteen Sixty-Eight
MM, MONTH, Month, MON, Mon, RM	02, FEBRUARY, February, FEB, Feb, II
WW, IW, W	06, 06, 1
DDD, DD, DAY, Day, DY, Dy, J	036, 05, MONDAY, Monday, MON, Mon, 2439892
ddTH, DDTH, ddSP, DDSP, DDSPTH	05th, 05TH, five, FIVE, FIFTH

Convertendo Data/Hora

- ***Exemplo:***
  - ***Converte*** **5** de ***fevereiro*** de **1968** em um ***string*** com o ***formato*** **MONTH DD, YYYY**

```
SELECT TO_CHAR(TO_DATE('05-FEB-1968'), 'MONTH DD, YYYY')
FROM dual;
```

TO_CHAR(TO_DATE('05-FEB-1968'),'MONTHDD,YYYY')
FEBRUARY 05, 1968

Convertendo Data/Hora

- **TO_DATE**(x, [formato])
  - ***Converte*** o ***string*** "x" em ***uma data/hora***
  - ***Permite fornecer*** o ***string opcional*** para ***indicar*** o ***formato*** de "x"
  - Se for ***omitido*** o ***formato***, a ***data*** será ***formatada*** no ***padrão*** do **BD**
  - ***Parâmetro*** **NLS_DATE_FORMAT** ***especifica*** o ***formato*** de ***data padrão*** do **SGBD** (**DD-MON-YY**)

Convertendo Data/Hora

- ***Exemplo:***
  - ***Converte*** os ***strings*** **04-JUL-2013** e **04-JUL-13** na ***data*** de **04-JUL-13**

```
SELECT TO_DATE('04-JUL-2013'), TO_DATE('04-JUL-13')
FROM dual;
```

TO_DATE('04-JUL-2013')	TO_DATE('04-JUL-13')
04-JUL-13	04-JUL-13

Formatando Data/Hora

- ***Possibilidade*** de ***utilização*** do ***formato opcional*** para uma ***data/hora*** (**TO_DATE**)
- ***Exemplo (01):***
  - ***Converte*** o ***string*** *"Jul 4, 2013"* em ***uma data***, ***passando*** o ***string*** de ***formato*** **MONTH DD, YYYY** para **TO_DATE**

```
SELECT TO_DATE('Jul 04, 2013', 'MONTH DD, YYYY')
FROM dual;

SELECT *
FROM nls_session_parameters;

ALTER SESSION SET NLS_DATE_FORMAT='Mon/dd/yyyy';
```

Formatando Data/Hora

- ***Exemplo (02):***
  - ***Converte*** o ***string*** 7.4.13 na ***data*** Jul 4, 2013
  - A ***data final*** é ***exibida*** no ***formato determinado*** pelo ***parâmetro*** **NLS_DATE_FORMAT**

```
SELECT TO_DATE('7.4.13', 'MM.DD.YY')
FROM dual;
```

TO_DATE('7.4.13','MM.DD.YY')
Jul/04/2013

Especificando Horas

- ***Possibilidade*** de ***especificar*** uma ***hora*** em uma ***data/hora***
- Se ***não*** for ***especificado*** um ***formato*** para ***hora***, por ***padrão***, será ***utilizado*** **12:00:00 A.M.**

Especificando Horas

- ***Exemplo (01):***
  - ***Adicionar*** um ***novo registro*** na **TB_CLIENTES**, ***utilizando*** a ***função*** **TO_DATE** na ***coluna*** **DT_NASCIMENTO**

```
INSERT INTO tb_clientes(id_cliente, nome, sobrenome, dt_nascimento,
                        telefone, fg_ativo)
VALUES (
10,
'Nome',
'Sobrenome',
 TO_DATE('Jul 04, 2013 19:32:36', 'MONTH DD, YYYY HH24:MI:SS'),
 '800-555-1215',
1);
```

Especificando Horas

- ***Exemplo (02):***
  - ***Recupera*** as ***linhas*** da **TB_CLIENTES** ***utilizando*** **TO_CHAR** para ***converter*** os ***valores*** da ***coluna*** **DT_NASCIMENTO**
  - ***Observe*** que o **ID_CLIENTE** de **nº 10** tem a ***hora configurada*** pela ***instrução*** **INSERT**

```
SELECT id_cliente,
       TO_CHAR(dt_nascimento, 'DD-MON-YYYY HH24:MI:SS')
FROM tb_clientes
ORDER BY id_cliente;

ROLLBACK;
```

Mesclando TO_CHAR e TO_DATE

- ***Permite mesclar chamadas*** de **TO_CHAR** e **TO_DATE**
- Desta forma, ***torna-se possível utilizar data/hora*** em ***diferentes formatos***
- ***Exemplo:***
  - ***Combinação*** de **TO_CHAR** e **TO_DATE** para ***exibir apenas*** o ***segmento referente*** à ***hora*** de uma ***data/hora***. A ***saída*** de **TO_DATE** é ***passada*** como ***parâmetro*** para **TO_CHAR**

```
SELECT TO_CHAR(TO_DATE('Jul 04, 2013 19:32:36',
                'MONTH DD, YYYY HH24:MI:SS'), 'HH24:MI:SS')
FROM dual;
```

TO_CHAR(TO_DATE('JUL04,201319:32:36','MONTHDD,YYYYHH24:MI:SS'),'HH24:MI:SS')
19:32:36

Configurando a Data Padrão

- O ***formato*** de ***data padrão*** é ***especificado*** pelo ***parâmetro*** **NLS_DATE_FORMAT**
- O **DBA** poderá ***alterar*** a ***configuração*** de **NLS_DATE_FORMAT** ***definindo*** um ***novo valor*** para esse ***parâmetro*** no ***arquivo*** init.ora ou spfile.ora
- ***Também*** é ***possível*** o **DBA** ***alterar*** essa ***configuração usando*** o ***comando*** **ALTER SYSTEM** ou ***alterar somente*** na ***sessão***, ***através*** do ***comando*** **ALTER SESSION**

Configurando a Data Padrão

- ***Observação:***
  - Uma ***sessão*** é ***iniciada*** quando ***você conecta*** a um **BD** e é ***finalizada*** quando ***você*** se ***desconecta***
- ***Exemplo:***
  - A ***instrução*** **ALTER SESSION** ***abaixo configura*** **NLS_DATE_FORMAT** como **Mon/DD/YYYY**

```
SELECT *
FROM nls_session_parameters;

ALTER SESSION SET NLS_DATE_FORMAT='Mon/dd/yyyy';
```

Configurando a Data Padrão

- ***Exemplo:***
  - A ***instrução*** **ALTER SESSION** ***abaixo configura*** **NLS_DATE_FORMAT** como **Mon/DD/YYYY**

```
SELECT *
FROM nls_session_parameters;

ALTER SESSION SET NLS_DATE_FORMAT='Mon/dd/yyyy';
```

PARAMETER	VALUE
NLS_LANGUAGE	AMERICAN
NLS_TERRITORY	AMERICA
NLS_CURRENCY	$
NLS_ISO_CURRENCY	AMERICA
NLS_NUMERIC_CHARACTERS	.,
NLS_CALENDAR	GREGORIAN
NLS_DATE_FORMAT	Mon/dd/yyyy
NLS_DATE_LANGUAGE	AMERICAN

Configurando a Data Padrão

- Se o ***formato*** do **BD** ***utiliza*** **YY** para o ***ano*** e ***você fornece apenas dois dígitos*** de ***um ano***, será ***presumido*** que o ***século*** do ***ano*** é o ***atual***, ***configurado*** no ***servidor*** do **BD**

Usando o Formato YY

- ***Exemplo:***
  - ***Observe*** as ***datas*** de ***entrada*** **15** e **75** as quais são ***passadas*** para **TO_DATE**, cuja ***saída*** é ***passada*** para **TO_CHAR**, que ***converte*** as ***datas*** em um ***string*** com o ***formato*** **DD-MON-YYYY**

```
SELECT
  TO_CHAR(TO_DATE('Jul 04, 15', 'MONTH DD, YY'), 'MONTH DD, YYYY'),
  TO_CHAR(TO_DATE('Jul 04, 75', 'MONTH DD, YY'), 'MONTH DD, YYYY')
FROM dual;
```

TO_CHAR(TO_DATE('JUL04,15','MONTHDD,YY'),'MONTHDD,YYYY')	TO_CHAR(TO_DATE('JUL04,75','MONTHDD,YY'),'MONTHDD,YYYY')
JULY 04, 2015	JULY 04, 2075

Usando o Formato RR

- ***Caso*** o ***formato*** da ***data*** seja **RR** e, se ***você fornecer*** os ***dois últimos dígitos*** de ***um ano***, os ***dois primeiros dígitos*** do ***ano*** são ***determinados usando*** o ***ano*** de ***dois dígitos fornecido***
- ***Como*** os ***anos*** de ***dois dígitos*** são ***interpretados:***

		Ano fornecido de dois dígitos	
		00–49	**50–99**
Dois últimos dígitos do ano atual	**00–49**	Regra 1: os dois primeiros dígitos do ano fornecido são definidos como os dois primeiros dígitos do ano atual.	Regra 2: os dois primeiros dígitos do ano fornecido são definidos como os dois primeiros dígitos do ano atual menos 1.
	50–99	Regra 3: os dois primeiros dígitos do ano fornecido são definidos como os dois primeiros dígitos do ano atual mais 1.	Regra 4: os dois primeiros dígitos do ano fornecido são definidos como os dois primeiros dígitos do ano atual.

Usando o Formato RR

- ***Exemplo:***
  - A ***consulta abaixo*** usa o ***formato*** **RR** para ***interpretar*** os ***anos*** **15** e **75** (ano ***atual*** **2013**)

```
SELECT
  TO_CHAR(TO_DATE('Jul 04, 15', 'MONTH DD, RR'), 'MONTH DD, YYYY'),
  TO_CHAR(TO_DATE('Jul 04, 75', 'MONTH DD, RR'), 'MONTH DD, YYYY')
FROM dual;
```

TO_CHAR(TO_DATE('JUL04,15','MONTHDD,RR'),'MONTHDD,YYYY')	TO_CHAR(TO_DATE('JUL04,75','MONTHDD,RR'),'MONTHDD,YYYY')
JULY 04, 2015	JULY 04, 1975

Funções de Data/Hora

- As ***funções*** de ***data/hora*** são ***usadas*** para ***obter*** ou ***processar data/hora*** e ***timestamp***
- A ***tabela abaixo apresenta algumas*** das ***funções*** de ***data/hora***

Function	Description
`ADD_MONTHS(x, y)`	Returns the result of adding *y* months to *x*. If *y* is negative, *y* months are subtracted from *x*.
`LAST_DAY(x)`	Returns the last day of the month part of *x*.
`MONTHS_BETWEEN(x, y)`	Returns the number of months between *x* and *y*. If *x* appears before *y* on the calendar, the number returned is positive; otherwise the number is negative.
`NEXT_DAY(x, day)`	Returns the datetime of the next day following *x*; *day* is specified as a literal string (`SATURDAY`, for example).

Funções de Data/Hora

- A ***tabela abaixo apresenta algumas*** das ***funções*** de ***data/hora*** (*continuação*):

Function	Description
`ROUND(x [, unit])`	Rounds `x`. By default, `x` is rounded to the beginning of the nearest day. You may supply an optional `unit` string that indicates the rounding unit; for example, `YYYY` rounds `x` to the first day of the nearest year.
`SYSDATE`	Returns the current datetime set in the database server's operating system.
`TRUNC(x [, unit])`	Truncates `x`. By default, `x` is truncated to the beginning of the day. You may supply an optional `unit` string that indicates the truncating unit; for example, `MM` truncates `x` to the first day of the month.

Funções de Data/Hora

- **ADD_MONTHS(x, y)**
  - ***Retorna*** o ***resultado*** da ***adição*** de "y" ***meses*** a "x"
  - ***Se*** "y" for ***negativo***, "y" ***meses*** são ***subtraídos*** de "x"
- ***Exemplo (01):***
  - ***Soma 13 meses*** à **1º** de ***julho*** de ***2013***

```
SELECT ADD_MONTHS('Jul 01, 2013', 13)
FROM dual;
```

ADD_MONTHS('JUL01,2013',13)
Aug/01/2014

Funções de Data/Hora

- **ADD_MONTHS(x, y)**
  - ***Retorna*** o ***resultado*** da ***adição*** de "y" ***meses*** a "x"
  - ***Se*** "y" for ***negativo***, "y" ***meses*** são ***subtraídos*** de "x"
- ***Exemplo (02):***
  - ***Subtrair 13 meses*** à **1º** de ***julho*** de ***2013***

```
SELECT ADD_MONTHS('Jul 01, 2013', -13)
FROM dual;
```

ADD_MONTHS('JUL01,2013',-13)
Jun/01/2012

Funções de Data/Hora

- **LAST_DAY(x)**
  - ***Retorna*** a ***data*** do ***último dia*** da ***parte*** de "x" ***referente*** ao ***mês***
- ***Exemplo:***
  - ***Exibir*** a ***última data*** em ***julho*** de **2013**

```
SELECT LAST_DAY('Jul 01, 2013')
FROM dual;
```

LAST_DAY('JUL01,2013')
Jul/31/2013

Funções de Data/Hora

- **MONTHS_BETWEEN**(x, y)
  - ***Retorna*** o ***número*** de ***meses entre*** "x" e "y"
  - ***Se*** "x" ***ocorre antes*** de "y" no ***calendário***, o ***número retornado*** pela ***função*** é ***negativo***
- ***Exemplo:***
  - ***Exibir*** o ***número*** de ***meses*** entre **1º** de ***maio*** de **2011** e **03** de ***julho*** de ***2013***

```
SELECT MONTHS_BETWEEN('Jul 03, 2013', 'May 01, 2011')
FROM dual;
```

MONTHS_BETWEEN('JUL03,2013','MAY01,2011')
26.064516129032258064516129032258064516 13

Funções de Data/Hora

- **NEXT_DAY**(x, dia)
  - ***Retorna*** a ***data*** do ***próximo dia depois*** de "x"
  - O ***dia*** é ***determinado*** como um ***string literal*** (***Exemplo:*** *Saturday*)
- ***Exemplo:***
  - ***Exibir*** a ***data*** do ***próximo domingo***, após **03** de ***julho*** de **2013**

```
-- Sunday = 1; Monday = 2; Tuesday = 3 ....
SELECT NEXT_DAY('Jul 03, 2013', 1)
FROM dual;
```

NEXT_DAY('JUL03,2013',1)
Jul/07/2013

Funções de Data/Hora

- **ROUND**(x, [unidade])
  - ***Arredonda*** "x", por ***padrão***, para o ***início*** do ***dia mais próximo***
  - ***Permite fornecer*** o ***string opcional***, "x" será ***arredondado*** para ***essa unidade***
  - ***String opcional:*** **YYYY *arredonda*** "x" para o ***primeiro dia*** do ***ano mais próximo***

Funções de Data/Hora

- **ROUND**(x, [unidade])
- ***Exemplo (01):***
  - ***Arredondar* 03** de ***julho*** de **2013** para o **1º *dia*** do ***ano*** mais ***próximo*** (**1º** de ***janeiro*** de **2014**)

```
SELECT ROUND(TO_DATE('Jul 03, 2013'), 'YYYY')
FROM dual;
```

ROUND(TO_DATE('JUL03,2013'),'YYYY')
Jan/01/2014

Funções de Data/Hora

- **ROUND**(x, [unidade])
- ***Exemplo (02):***
  - ***Arredondar* 25** de ***maio*** de **2013** para o **1º *dia*** do ***mês mais próximo*** (**1º** de ***junho*** de **2013**)

```
SELECT ROUND(TO_DATE('May 25, 2013'), 'MM')
FROM dual;
```

ROUND(TO_DATE('MAY25,2013'),'MM')
Jun/01/2013

Funções de Data/Hora

- **ROUND**(x, [unidade])
- ***Exemplo (03):***
  - ***Arredondar* 19:45:26 *horas*** de **03** de ***julho*** de **2013** para a ***hora mais próxima*** (**20:00**h)

```
SELECT TO_CHAR
        (ROUND
        (TO_DATE('Jul 03, 2013 19:45:26', 'MONTH DD, YYYY HH24:MI:SS'),
        'HH24'),
        'MONTH DD, YYYY HH24:MI:SS')
FROM dual;
```

TO_CHAR(ROUND(TO_DATE('JUL03,201319:45:26','MONTHDD,YYYYHH24:MI:SS'),'HH24'),'MONTHDD,YYYYHH24:MI:SS')
JULY 03, 2013 20:00:00

Funções de Data/Hora

- **SYSDATE**
  - ***Retorna data/hora atual configurada*** no **SO** do ***servidor*** do **BD**
- ***Exemplo:***
  - ***Obter*** a ***data atual***

```
SELECT SYSDATE
FROM dual;
```

SYSDATE
Jul/03/2013

Funções de Data/Hora

- **EXTRACT**
  - ***Extrai*** e ***retorna*** o ***ano***, ***mês***, ***dia***, ***hora***, ***minuto***, ***segundo*** ou ***fuso horário*** de "x"
  - "x" ***pode ser*** um **TIMESTAMP** ou um ***tipo*** **DATE**

Funções de Data/Hora

- **EXTRACT**
- ***Exemplo (01):***
  - ***Obter*** o ***ano***, ***mês*** e ***dia*** de um ***tipo*** **DATE** ***retornado*** por **TO_DATE**

```
SELECT
  EXTRACT(YEAR FROM TO_DATE('Jul 03, 2013 19:45:26',
     'MONTH DD, YYYY HH24:MI:SS')) AS YEAR,
  EXTRACT(MONTH FROM TO_DATE('Jul 03, 2013 19:45:26',
     'MONTH DD, YYYY HH24:MI:SS')) AS MONTH,
  EXTRACT(DAY FROM TO_DATE('Jul 03, 2013 19:45:26',
     'MONTH DD, YYYY HH24:MI:SS')) AS DAY
FROM dual;
```

YEAR	MONTH	DAY
2013	7	3

Funções de Data/Hora

- **EXTRACT**
- ***Exemplo (02):***
  - ***Obter*** a ***hora***, ***minuto*** e ***segundo*** de ***um valor*** **TIMESTAMP** ***retornado*** por **TIMESTAMP**

```
SELECT
 EXTRACT(HOUR FROM TO_TIMESTAMP('Jul 03, 2013 19:45:26',
    'MONTH DD, YYYY HH24:MI:SS')) AS HORA,
 EXTRACT(MINUTE FROM TO_TIMESTAMP('Jul 03, 2013 19:45:26',
    'MONTH DD, YYYY HH24:MI:SS')) AS MINUTO,
 EXTRACT(SECOND FROM TO_TIMESTAMP('Jul 03, 2013 19:45:26',
    'MONTH DD, YYYY HH24:MI:SS')) AS SEGUNDO
FROM dual;
```

HORA	MINUTO	SEGUNDO
19	45	26

Intervalo de Tempo

- O ***Oracle 9i introduziu tipos*** de ***dados*** que ***permitem armazenar intervalos*** de ***tempo***
- ***Exemplos*** de ***intervalos*** de ***tempo incluem:***
  - ***1 ano*** de ***3 meses***
  - ***25 meses***
  - ***1 dia*** e ***7 horas***
  - ***-56 horas***
- Em ***nosso esquema*** **LOJA** é ***possível fornecer descontos*** para os ***produtos*** por ***tempo limitado***

Intervalo de Tempo

- ***Usando*** o ***tipo*** **INTERVAL YEAR TO MONTH**
- ***Armazena*** um ***intervalo*** de ***tempo medido*** em ***anos*** e ***meses***
- ***Tipos*** e ***Intervalo:***

Type	Description
`INTERVAL YEAR[ (years_precision) ] TO MONTH`	Stores a time interval measured in years and months. You can specify an optional precision for the years using *years_precision*, which may be an integer from 0 to 9. The default precision is 2, which means you can store two digits for the years in your interval. If you try to add a row with more year digits than your `INTERVAL YEAR TO MONTH` column can store, you'll get an error. You can store a positive or negative time interval.
`INTERVAL DAY[ (days_precision) ] TO SECOND[ (seconds_precision) ]`	Stores a time interval measured in days and seconds. You can specify an optional precision for the days using *days_precision* (an integer from 0 to 9; the default is 2). In addition, you can also specify an optional precision for the fractional seconds using *seconds_precision* (an integer from 0 to 9; the default is 6). You can store a positive or negative time interval.

Intervalo de Tempo

- A ***tabela*** **TB_PROMOCAO** ***contém*** a ***coluna*** **INTERVAL YEAR TO MONTH** ***nomeada*** de **DURACAO** para ***registrar*** o ***intervalo*** de ***tempo durante*** a ***validade*** do ***cupom***

```
SELECT *
FROM tb_promocao;
```

ID_PROMOCAO	NOME	DURACAO
1	10% off Z Files	3 0:0:0.0
2	20% off Pop 3	0 2:0:0.0
3	30% off Modern Science	0 0:25:0.0
4	20% off Tank War	0 0:0:45.0
5	10% off Chemistry	3 2:25:0.0
6	20% off Creative Yell	3 2:25:45.0
7	15% off My Front Line	123 2:25:45.12

Intervalo de Tempo

- Para ***fornecer*** um ***valor*** **INTERVAL YEAR TO MONTH** ***literal*** para o **BD**, use a ***seguinte sintaxe:***

  INTERVAL '[**+/-**][**y**][**m**]' [**YEAR**[(*precisão_anos*)])] [**TO MONTH**]

- ***Onde:***
  - **+** ou **–** é um ***indicador opcional*** que ***específica*** se o ***intervalo*** de ***tempo*** é **positivo** ou ***negativo*** (***padrão*** é ***positivo***)
  - "**y**" é o ***número*** de ***anos opcional*** para o ***intervalo*** (**TO YEAR**)
  - "**m**" é o ***número*** de ***meses opcional*** para o ***intervalo*** (**TO MONTH**)
  - "**precisão_anos**" é a ***precisão opcional*** para os ***anos*** (***padrão*** é **2**)

Intervalo de Tempo

- A ***tabela abaixo apresenta*** alguns ***exemplos*** de ***literais*** de ***intervalo*** de ***ano*** para ***mês***

Literal	Description
`INTERVAL '1' YEAR`	Interval of 1 year.
`INTERVAL '11' MONTH`	Interval of 11 months.
`INTERVAL '14' MONTH`	Interval of 14 months (equivalent to 1 year 2 months).
`INTERVAL '1-3' YEAR TO MONTH`	Interval of 1 year 3 months.
`INTERVAL '0-5' YEAR TO MONTH`	Interval of 0 years 5 months.
`INTERVAL '123' YEAR(3) TO MONTH`	Interval of 123 years with a precision of 3 digits.
`INTERVAL '-1-5' YEAR TO MONTH`	A negative interval of 1 year 5 months.
`INTERVAL '1234' YEAR(3)`	Invalid interval: 1234 contains four digits and therefore contains one too many digits allowed by the precision of 3 (three digits maximum).

Intervalo de Tempo

- Para ***fornecer*** um ***valor*** **INTERVAL DAY TO SECOND** ***literal*** para o **BD**, use a ***seguinte sintaxe:***

  INTERVAL '[**+/-**][**d**][**h**[:**m**[:**s**]]]' [**DAY**[(*precisão_dias*)]]) [**TO HOUR** | MINUTE | SECOND[(precisão_segundos)]]

- ***Onde:***
  - **+** ou **–** é um ***indicador opcional*** que ***específica*** se o ***intervalo*** de ***tempo*** é **positivo** ou ***negativo*** (***padrão*** é ***positivo***)
  - "**d**" é o ***número*** de ***dias*** do ***intervalo***
  - "**h**" é o ***número*** de ***horas opcional*** para o ***intervalo*** (**TO HOUR**)
  - "**m**" é o ***número*** de ***minutos opcional*** do ***intervalo*** (**TO MINUTE**)

Intervalo de Tempo

- Para ***fornecer*** um ***valor*** **INTERVAL DAY TO SECOND** ***literal*** para o **BD**, use a ***seguinte sintaxe:***

  INTERVAL '[**+/-**][**d**][**h**[:**m**[:**s**]]]' [**DAY**[(*precisão_dias*)]]) [**TO HOUR** | MINUTE | SECOND[(precisão_segundos)]]

- ***Onde*** (*continuação*)***:***
  - "**s**" é o ***número*** de ***segundos opcional*** do ***intervalor*** (**TO SECOND**)
  - "**precisão_dias**" é a ***precisão opcional*** dos ***dias*** (***padrão*** é **2**)
  - "**precisão_segundos**" é a ***precisão opcional*** dos ***segundos fracionários*** (***padrão*** é **6**)

Intervalo de Tempo

- A ***tabela abaixo*** apresenta ***alguns exemplos*** de ***literais*** de ***intervalo*** de ***dias*** até ***segundos***

Literal	Description
`INTERVAL '3' DAY`	Interval of 3 days.
`INTERVAL '2' HOUR`	Interval of 2 hours.
`INTERVAL '25' MINUTE`	Interval of 25 minutes.
`INTERVAL '45' SECOND`	Interval of 45 seconds.
`INTERVAL '3 2' DAY TO HOUR`	Interval of 3 days 2 hours.
`INTERVAL '3 2:25' DAY TO MINUTE`	Interval of 3 days 2 hours 25 minutes.
`INTERVAL '3 2:25:45' DAY TO SECOND`	Interval of 3 days 2 hours 25 minutes 45 seconds.
`INTERVAL '123 2:25:45.12' DAY(3) TO SECOND(2)`	Interval of 123 days 2 hours 25 minutes 45.12 seconds; the precision for days is 3 digits, and the precision for the fractional seconds is 2 digits.
`INTERVAL '3 2:00:45' DAY TO SECOND`	Interval of 3 days 2 hours 0 minutes 45 seconds.
`INTERVAL '-3 2:25:45' DAY TO SECOND`	Negative interval of 3 days 2 hours 25 minutes 45 seconds.
`INTERVAL '1234 2:25:45' DAY(3) TO SECOND`	Invalid interval because the number of digits in the days exceeds the specified precision of 3.
`INTERVAL '123 2:25:45.123' DAY TO SECOND(2)`	Invalid interval because the number of digits in the fractional seconds exceeds the specified precision of 2.

Intervalo de Tempo

- ***Funções*** de ***Intervalo*** de ***Tempo***
  - ***Permitem obter*** e ***processar intervalos*** de ***tempo***

- **NUMTODSINTERVAL**(*x*, *unidade_intervalo*)
  - ***Converte*** o ***número*** *"x"* em um ***valor INTERVAL DAY TO SECOND***
  - O ***intervalo*** de *"x"* é ***especificado*** em *unidade_intervalo*, que pode ser **DAY**, **HOUR**, **MINUTE** ou **SECOND**

Intervalo de Tempo

- ***Exemplo (01):***
  - ***Converter vários números*** em ***intervalos*** de ***tempo*** usando *NUMTODSINTERVAL*

```
SELECT
  NUMTODSINTERVAL(1.5,'DAY'),
  NUMTODSINTERVAL(3.25,'HOUR'),
  NUMTODSINTERVAL(5,'MINUTE'),
  NUMTODSINTERVAL(10.123456789,'SECOND')
FROM dual;
```

NUMTODSINTERVAL(1.5,'DAY')	NUMTODSINTERVAL(3.25,'HOUR')	NUMTODSINTERVAL(5,'MINUTE')	NUMTODSINTERVAL(10.123456789,'SECOND')
1 12:0:0.0	0 3:15:0.0	0 0:5:0.0	0 0:0:10.123456789

Intervalo de Tempo

- ***Funções*** de ***Intervalo*** de ***Tempo***
  - ***Permitem obter*** e ***processar intervalos*** de ***tempo***

- **NUMTOYMINTERVAL**(*x*, *unidade_intervalo*)
  - ***Converte*** o ***número*** *"x"* em um ***valor INTERVAL YEAR TO MONTH***
  - O ***intervalo*** de *"x"* é ***especificado*** em *unidade_intervalo*, que pode ser **YEAR** ou **MONTH**

Intervalo de Tempo

- ***Exemplo (01):***
  - ***Converter dois números*** em ***intervalos*** de ***tempo usando*** *NUMTOYMINTERVAL*

```
SELECT
  NUMTOYMINTERVAL(1.5,'YEAR'),
  NUMTOYMINTERVAL(3.25,'MONTH')
FROM dual;
```

NUMTOYMINTERVAL(1.5,'YEAR')	NUMTOYMINTERVAL(3.25,'MONTH')
1-6	0-3

Subconsultas

Subconsultas

- ***Objetivos:***
  - ***Como inserir*** uma ***instrução*** **SELECT** ***interna dentro*** de uma ***instrução*** **SELECT**, **UPDATE** ou **DELETE** ***externa*** (a ***instrução*** **SELECT** ***interna*** é ***conhecida*** como ***subconsulta***)
  - As ***principais características*** dos ***diferentes tipos*** de ***subconsultas***
  - ***Como*** as ***subconsultas permitem construir instruções complexas*** a ***partir*** de ***componentes simples***

Subconsultas

- ***Basicamente, existem dois tipos*** de ***subconsultas:***
  - **Única linha:** ***retornam zero*** ou ***uma linha*** para a ***instrução*** **SQL** ***externa***. ***Existe*** um ***caso especial*** de ***subconsulta*** de uma ***única linha*** que ***contém exatamente*** uma ***coluna***, esse ***tipo*** de ***consulta*** é ***chamada*** de ***subconsulta escalar***
  - **Várias linhas:** ***retornam uma*** ou ***mais linhas*** para a ***instrução*** **SQL** ***externa***

Subconsultas

- ***Além disso***, existem ***três tipos*** de ***subconsultas*** que podem ***retornar uma*** ou ***várias linhas:***
  - **Correlacionadas:** ***referenciam uma*** ou ***mais colunas*** na ***instrução* SQL *externa***. Elas ***são chamadas*** de ***subconsultas*** "***correlacionadas***" porque são ***relacionadas*** à ***instrução* SQL *externa*** por ***meio*** das ***mesmas colunas***
  - **Aninhadas:** ***inseridas dentro*** de ***outra subconsulta***. É ***permitido aninhar subconsultas*** até uma ***profundidade*** de **255**

Subconsultas de uma única linha

Subconsultas (única linha)

- É ***aquela*** que ***retorna zero*** ou ***uma linha*** para a ***instrução*** **SQL** ***externa***
- Pode ser ***inserida*** em uma ***cláusula*** **WHERE**, **HAVING** ou **FROM** de uma ***instrução*** **SELECT**
- ***Exemplo*** (*cláusula* ***WHERE***)***:***
  - ***Recuperar*** os ***valores*** de **NOME** e **SOBRENOME** da **TB_CLIENTES**, cujo ***valor*** do **SOBRENOME** seja "***Blue***"

```
SELECT nome, sobrenome
FROM tb_clientes
WHERE id_cliente = (SELECT id_cliente
                    FROM tb_clientes
                    WHERE sobrenome = 'Blue');
```

Subconsultas (única linha)

- ***Vamos analisar*** essa ***consulta***, a ***subconsulta*** na ***cláusula*** **WHERE** é:

```
SELECT id_cliente
FROM tb_clientes
WHERE sobrenome = 'Blue';
```

- Essa ***subconsulta*** é ***executada primeiro*** (e ***apenas uma vez***) e ***retorna*** o **ID_CLIENTE** da ***coluna*** cujo ***valor*** do **SOBRENOME** seja *"Blue"*
- O ***valor*** do **ID_CLIENTE** para essa ***linha*** é **5**, o ***qual*** é ***passado*** para a ***cláusula*** **WHERE** da ***consulta externa***

Subconsultas (única linha)

- ***Usando outros operadores*** de ***uma única linha***
  - O ***exemplo anterior*** foi ***utilizado*** o ***operador*** de ***igualdade*** (**=**) na ***cláusula*** **WHERE**
  - ***Outros operadores*** de ***comparação*** (**<>**, **<**, **>**, **<=** e **>=**) ***podem*** ser ***utilizados*** em uma ***subconsulta*** de ***uma única linha***

Subconsultas (única linha)

- ***Exemplo:***
  - ***Obter*** o ***preço médio*** dos ***produtos***, o qual é ***passado*** para a ***cláusula*** **WHERE** da ***consulta externa***. A ***consulta externa retorna*** os ***valores*** de **ID_PRODUTO**, **NM_PRODUTO** e **PRECO** dos ***produtos*** cujo ***preço*** é ***maior*** do que a ***média*** de ***preços***

```
SELECT id_produto, nm_produto, preco
FROM tb_produtos
WHERE preco > (SELECT AVG(preco)
               FROM tb_produtos);
```

ID_PRODUTO	NM_PRODUTO	PRECO
6	Z Files	49.99
1	Modern Science	19.95
2	Chemistry	30
3	Supernova	25.99

Subconsultas (única linha)

- O ***exemplo*** a ***seguir apresenta*** a ***decomposição*** da ***consulta anterior***, ***executando*** a ***subconsulta separadamente***

```
SELECT AVG(preco)
FROM tb_produtos;
```

AVG(PRECO)
19.730833333333333333333333333333333333333

- ***Obs.:*** essa ***subconsulta*** é um ***exemplo*** de ***subconsulta escalar***, pois ***retorna exatamente uma linha contendo*** uma ***coluna***. O ***valor retornado*** por ***uma subconsulta escalar*** é ***tratado*** como um ***único valor escalar***

Subconsultas (única linha)

- ***Subconsulta*** em uma ***cláusula*** **HAVING**
  - **HAVING:** ***usada*** para ***filtrar grupos*** de ***linhas***
  - ***Permite inserir*** uma ***subconsulta*** na ***cláusula*** **HAVING** de uma ***consulta externa***
  - Possibilita ***filtrar grupos*** de ***linhas*** com ***base*** no ***resultado retornado*** por sua ***subconsulta***

Subconsultas (única linha)

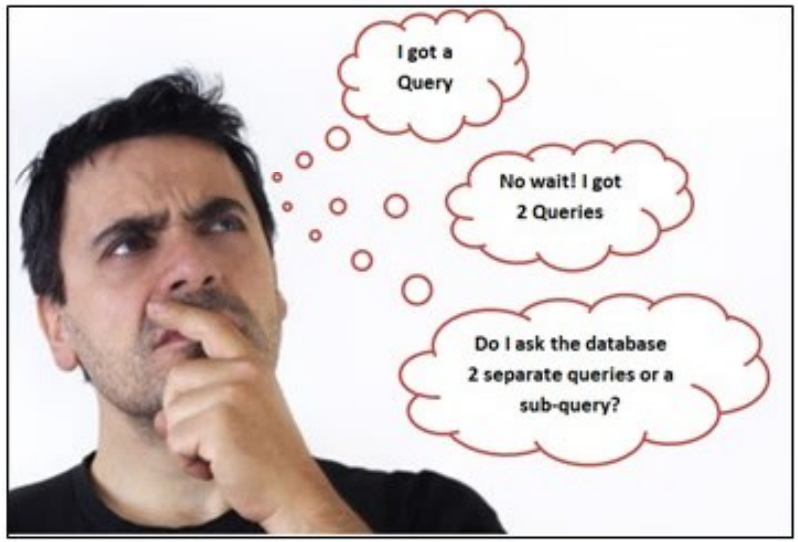
- ***Subconsulta*** em uma ***cláusula*** **HAVING**
- **Exemplo:**
  - ***Recupera*** o ***valor*** do **ID_TIPO_PRODUTO** e o ***preço médio*** dos ***produtos*** cujo ***preço médio*** é ***menor*** do que o ***máximo*** da ***média*** dos ***grupos*** do ***mesmo tipo*** de ***produto***

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
GROUP BY id_tipo_produto
HAVING AVG(preco) < (SELECT MAX(AVG(preco))
                     FROM tb_produtos
                     GROUP BY id_tipo_produto)
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	AVG(PRECO)
1	24.975
3	13.24
4	13.99
(null)	13.49

Subconsultas (única linha)

- ***Subconsulta*** em uma ***cláusula*** **FROM** (visões ***inline***)
  - ***Permite inserir*** uma ***subconsulta*** na ***cláusula*** **FROM** de uma ***consulta externa***
  - Esses ***tipos*** de ***subconsultas*** também são ***conhecidas*** como ***visões inline***



Subconsultas (única linha)

- ***Subconsulta*** em uma ***cláusula*** **FROM** (visões ***inline***)
- ***Exemplo (01):***
  - ***Recuperar*** os ***produtos*** cujo ***valor*** do **ID_PRODUTO** seja ***menor*** do que **3**

```
SELECT id_produto
FROM (SELECT id_produto
      FROM tb_produtos
      WHERE id_produto < 3);
```

ID_PRODUTO
1
2

Subconsultas (única linha)

- ***Subconsulta*** em uma ***cláusula*** **FROM** (***visões inline***)

```
SELECT id_produto
FROM (SELECT id_produto
      FROM tb_produtos
      WHERE id_produto < 3);
```

ID_PRODUTO
1
2

- No que ***diz respeito*** à ***cláusula*** **FROM** da ***consulta externa***, a ***saída*** da ***subconsulta*** é ***apenas*** outra ***fonte*** de ***dados***

Subconsultas (única linha)

- ***Subconsulta*** em uma ***cláusula*** **FROM** (***visões inline***)
- ***Exemplo (02):***
  - ***Recuperar*** os ***valores*** de **ID_PRODUTO** e **PRECO** da **TB_PRODUTOS** na ***consulta externa*** e a ***subconsulta recupera*** o ***número*** de ***vezes*** que um ***produto*** foi ***comprado***

```
SELECT p.id_produto, preco, dados_compra.count_produto
FROM tb_produtos p, (SELECT id_produto, COUNT(id_produto) count_produto
                     FROM tb_compras
                     GROUP BY id_produto) dados_compra
WHERE p.id_produto = dados_compra.id_produto;
```

ID_PRODUTO	PRECO	COUNT_PRODUTO
1	19.95	2
2	30	1
3	25.99	1
4	13.95	1

Subconsultas (única linha)

- ***Subconsulta*** em uma ***cláusula*** **FROM** (***visões inline***)
- ***Erros*** que ***podemos encontrar:***
  - As ***subconsultas*** de ***uma única linha*** podem ***retornar*** no ***máximo uma linha***
  - As ***subconsultas*** ***não*** podem ***conter*** uma ***cláusula*** **ORDER BY**

Subconsultas de várias linhas

Subconsultas (várias linhas)

- Uma ***consulta*** de ***várias linhas retorna uma*** ou ***mais linhas*** para uma ***instrução*** **SQL** ***externa***
- Os ***operadores*** **IN**, **ANY** e **ALL** são ***utilizados*** para ***realizar*** o ***tratamento*** de uma ***subconsulta*** que ***retorna várias linhas***
- ***Certificar*** se o ***valor*** de ***uma coluna*** está ***contido*** em ***uma lista*** de ***valores***
- A ***lista*** de ***valores*** pode ***vir*** de uma ***subconsulta***

Subconsultas (várias linhas)

- ***Exemplo:***

```
SELECT id_produto, nm_produto
FROM tb_produtos
WHERE id_produto IN (1, 2, 3);
```

ID_PRODUTO	NM_PRODUTO
1	Modern Science
2	Chemistry
3	Supernova

Subconsultas (várias linhas)

- ***Usando* IN** em uma ***subconsulta*** de ***várias linhas***
  - ***Uma lista*** de ***valores*** pode ***vir*** dos ***resultados retornados*** por uma ***subconsulta***
  - ***Permite utilizar* NOT** para ***executar*** a ***lógica oposta*** de **IN**

Subconsultas (várias linhas)

- ***Exemplo (01):***
  - ***Verificar*** se um ***valor*** de **ID_PRODUTO** se ***encontra*** na ***lista*** de ***valores retornada*** pela ***subconsulta***

```
SELECT id_produto, nm_produto
FROM tb_produtos
WHERE id_produto IN (SELECT id_produto
                     FROM tb_produtos
                     WHERE nm_produto LIKE '%e%');
```

ID_PRODUTO	NM_PRODUTO
6	Z Files
7	2412: The Return
8	Space Force 9
9	From Another Planet
12	Creative Yell
13	My Front Line
1	Modern Science
2	Chemistry
3	Supernova

Subconsultas (várias linhas)

- ***Usando* NOT IN** em uma ***subconsulta*** de ***várias linhas***
- ***Exemplo (02):***
  - ***Obter*** os ***produtos*** que ***não estão*** na **TB_COMPRAS**

```
SELECT id_produto, nm_produto
FROM tb_produtos
WHERE id_produto NOT IN (SELECT id_produto
                         FROM tb_compras);
```

ID_PRODUTO	NM_PRODUTO
13	My Front Line
8	Space Force 9
12	Creative Yell
10	Classical Music
11	Pop 3
6	Z Files
7	2412: The Return
9	From Another Planet

Subconsultas (várias linhas)

- ***Usando* ANY** em uma ***subconsulta*** de ***várias linhas***
  - O ***operador* ANY** é ***utilizado*** para ***comparar*** um ***valor*** com **QUALQUER** valor ***presente*** em ***uma lista***
  - ***Permite inserir*** um ***operador*** (**=, <>, <, >, <=** ou **>=**) ***antes*** de **ANY**

Subconsultas (várias linhas)

- ***Usando* ANY** em uma ***subconsulta*** de ***várias linhas***
- ***Exemplo:***
  - ***Obter*** os ***funcionários*** cujo ***salário*** seja ***menor*** do que **QUALQUER** um dos ***salários mais baixos*** da **TB_GRADES_SALARIOS**

```
SELECT id_funcionario, nome, salario
FROM tb_funcionarios
WHERE salario < ANY (SELECT base_salario
                     FROM tb_grades_salarios);
```

BASE_SALARIO
1
250001
500001
750001

Subconsultas (várias linhas)

- ***Usando* ALL** em uma ***subconsulta*** de ***várias linhas***
  - O ***operador* ALL** é ***utilizado*** para ***comparar*** um ***valor*** com **TODOS** os ***valores presentes*** em ***uma lista***
  - ***Permite inserir*** um ***operador*** (**=, <>, <, >, <=** ou **>=**) ***antes*** de **ALL**

Subconsultas (várias linhas)

- ***Usando* ALL** em uma ***subconsulta*** de ***várias linhas***
- ***Exemplo:***
  - ***Obter*** os ***funcionários*** cujo ***salário*** seja ***superior*** a **TODOS** os ***salários*** mais ***alto*** da **TB_GRADES_SALARIOS**

```
SELECT id_funcionario, nome, salario
FROM tb_funcionarios
WHERE salario > ALL (SELECT teto_salario
                     FROM tb_grades_salarios);
```

TETO_SALARIO
250000
500000
750000
999999

Subconsultas (várias linhas)

- ***Possibilidade*** de ***escrever subconsultas*** que ***retornam várias colunas***
- ***Exemplo:***
  - ***Recuperar*** os ***produtos*** com o ***menor preço*** para ***cada grupo*** de ***tipo*** de ***produto***
  - É ***retornado*** o ***valor*** do **ID_TIPO_PRODUTO** e o ***valor*** do **PRECO** ***mínimo*** para ***cada grupo*** de ***produtos***, onde os ***mesmos*** são ***comparados*** com os ***valores*** **ID_TIPO_PRODUTO** e **PRECO** para ***cada produto*** na ***cláusula*** **WHERE** da ***consulta externa***

Subconsultas (várias linhas)

- ***Possibilidade*** de ***escrever subconsultas*** que ***retornam várias colunas***
- ***Exemplo:***

```
SELECT id_produto, id_tipo_produto, nm_produto, preco
FROM tb_produtos
WHERE (id_tipo_produto, preco) IN (SELECT id_tipo_produto, MIN(preco)
                                   FROM tb_produtos
                                   GROUP BY id_tipo_produto);
```

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	PRECO
9	3	From Another Planet	12.99
10	4	Classical Music	10.99
1	1	Modern Science	19.95
4	2	Tank War	13.95

Subconsultas Correlacionadas

Subconsultas Correlacionadas

- Uma ***consulta correlacionada referencia uma*** ou ***mais colunas*** na ***instrução*** **SQL** ***externa***
- São ***nomeadas*** de ***correlacionadas*** pois são ***relacionadas*** à ***instrução*** **SQL** ***externa*** por meio das ***mesmas colunas***
- Uma ***consulta correlacionada*** é ***executada uma vez*** para ***cada linha*** na ***consulta externa***, diferenciando-se da ***subconsulta não correlacionada***, a qual é ***executada apenas uma vez antes*** da ***execução*** da ***consulta externa***
- Também ***permite trabalhar*** com ***valores nulos***

Subconsultas Correlacionadas

- ***Exemplo:***
  - ***Recuperar todos*** os ***produtos*** que ***possuem preço maior*** do que a ***média*** para o ***tipo*** de ***produto***

```
SELECT id_produto, id_tipo_produto, nm_produto, preco
FROM tb_produtos externa
WHERE preco > (SELECT AVG(preco)
               FROM tb_produtos interna
               WHERE interna.id_tipo_produto = externa.id_tipo_produto);
```

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	PRECO
6	2	Z Files	49.99
8	3	Space Force 9	13.49
11	4	Pop 3	15.99
12	4	Creative Yell	14.99
2	1	Chemistry	30

Subconsultas Correlacionadas

- ***Utilizamos*** o ***apelido*** "externa" para ***rotular*** a ***consulta externa*** e o ***apelido*** "interna" para a ***subconsulta interna***

- A ***referência*** à ***coluna*** **ID_TIPO_PRODUTO** nas ***partes interna*** e ***externa*** é que ***torna*** a ***subconsulta interna correlacionada*** à ***consulta externa***

- Em uma ***subconsulta correlacionada***, ***cada linha*** da ***consulta externa*** é ***passada*** por vez para a ***subconsulta***

Subconsultas Correlacionadas

- A ***subconsulta lê*** uma ***linha*** de ***cada vez*** da ***consulta externa*** e a ***aplica*** na ***subconsulta*** até que ***todas*** as ***linhas*** da ***consulta externa*** sejam ***processadas***

- No ***exemplo anterior***, a ***consulta externa recupera cada linha*** da **TB_PRODUTOS** e ***passa*** para a ***consulta interna***

- ***Cada linha*** é ***lida*** pela ***consulta interna***, a ***qual calcula*** o ***preço médio*** de ***cada produto*** onde o ***valor*** de **ID_TIPO_PRODUTO** na ***consulta interna*** é **IGUAL** ao ***valor*** de **ID_TIPO_PRODUTO** na ***consulta externa***

Subconsultas Correlacionadas

- ***Usando* EXISTS** e **NOT EXISTS** em uma ***subconsulta correlacionada***
  - O ***operador* EXISTS *verifica*** a ***existência*** de ***linhas retornadas*** por uma ***subconsulta***
  - ***Embora*** você possa ***usar* EXISTS** em ***subconsultas não correlacionadas***, em ***geral*** ele é ***utilizado*** em ***subconsultas correlacionadas***
  - **NOT EXISTS *executa*** a ***lógica oposta*** de **EXISTS**

Subconsultas Correlacionadas

- ***Usando* EXISTS** em uma ***subconsulta correlacionada***
- ***Exemplo:***
  - ***Recuperar*** os ***funcionários*** que ***gerenciam outros funcionários***

```
SELECT id_funcionario, nome, sobrenome
FROM tb_funcionarios externa
WHERE EXISTS (SELECT id_funcionario
              FROM tb_funcionarios interna
              WHERE interna.id_gerente = externa.id_funcionario);
```

ID_FUNCIONARIO	NOME	SOBRENOME
2	Ron	Johnson
4	Susan	Jones
7	John	Grey
6	Jane	Brown
1	James	Smith
10	Kevin	Black

Subconsultas Correlacionadas

- ***Como* EXISTS** apenas ***verifica*** a ***existência*** de ***linhas retornadas*** pela ***subconsulta***, uma ***subconsulta não*** precisa ***retornar*** uma ***coluna*** – ela pode ***retornar*** um ***valor literal***

- Essa ***característica*** pode ***melhorar*** o ***desempenho*** da ***consulta***

Subconsultas Correlacionadas

- ***Usando* EXISTS** em uma ***subconsulta correlacionada***
- ***Exemplo:***
  - ***Reescrever*** o ***exemplo anterior*** com a ***subconsulta retornando*** o ***valor literal*** **1**

```
SELECT id_funcionario, nome, sobrenome
FROM tb_funcionarios externa
WHERE EXISTS (SELECT 1
              FROM tb_funcionarios interna
              WHERE interna.id_gerente = externa.id_funcionario);
```

ID_FUNCIONARIO	NOME	SOBRENOME
2	Ron	Johnson
4	Susan	Jones
7	John	Grey
6	Jane	Brown
1	James	Smith
10	Kevin	Black

Subconsultas Correlacionadas

```
SELECT id_funcionario, nome, sobrenome
FROM tb_funcionarios externa
WHERE EXISTS (SELECT 1
              FROM tb_funcionarios interna
              WHERE interna.id_gerente = externa.id_funcionario);
```

- ***Independentemente*** da ***subconsulta retornar uma*** ou ***mais linhas***, **EXISTS** ***retornará*** **VERDADEIRO**
- Se a ***subconsulta não retornar uma linha***, **EXISTS** ***retornará*** **FALSO**

Subconsultas Correlacionadas

- ***Usando* NOT EXISTS** em ***uma subconsulta correlacionada***
- ***Exemplo:***
  - ***Recuperar*** os ***produtos*** que ***não*** foram ***comprados***

```
SELECT id_produto, nm_produto
FROM tb_produtos externa
WHERE NOT EXISTS (SELECT 1
                  FROM tb_compras interna
                  WHERE interna.id_produto = externa.id_produto);
```

ID_PRODUTO	NM_PRODUTO
13	My Front Line
8	Space Force 9
12	Creative Yell
10	Classical Music
11	Pop 3
6	Z Files
7	2412: The Return
9	From Another Planet

Subconsultas (Observação)

Subconsultas

- **EXISTS** e **NOT EXISTS** *versus* **IN** e **NOT IN**
  - O ***operador*** **IN** é ***usado*** para ***verificar*** se um ***valor está contido*** em ***uma lista***
  - **EXISTS** ***verifica apenas*** a ***existência*** de ***linhas***, ***enquanto*** que **IN** ***verifica*** os ***valores reais***
- *Observação:*
  - Normalmente, **EXISTS** ***oferece*** um ***desempenho melhor*** do que **IN** em ***subconsultas***

Subconsultas

- ***CUIDADO:***
  - Ao ***escrever consultas*** que ***utilizam*** **NOT EXISTS** ou **NOT IN**
  - ***Quando*** a ***lista*** de ***valores contém*** um ***valor nulo***, **NOT EXISTS** ***retorna*** **VERDADEIRO**, mas **NOT IN** ***retorna*** **FALSO**

Subconsultas

- ***Exemplo (01):***
  - ***Recuperar*** os ***tipos*** de ***produtos*** que **NÃO** têm ***produtos*** desse ***tipo*** na **TB_PRODUTOS**

```
SELECT id_tipo_produto, nm_tipo_produto
FROM tb_tipos_produtos externa
WHERE NOT EXISTS (SELECT 1
                  FROM tb_produtos interna
                  WHERE interna.id_tipo_produto = externa.id_tipo_produto);
```

ID_TIPO_PRODUTO	NM_TIPO_PRODUTO
5	Magazine

Subconsultas

- ***Exemplo (02):***
  - ***Reescrever*** a ***consulta anterior*** para ***utilizar*** o **NOT IN**
  - ***Observe*** que ***nenhuma linha*** é ***retornada***

```
SELECT id_tipo_produto, nm_tipo_produto
FROM tb_tipos_produtos
WHERE id_tipo_produto NOT IN (SELECT id_tipo_produto
                              FROM tb_produtos);
```

Subconsultas

Qual seria a ***solução adequada*** para o ***uso*** de **NOT IN**?

Subconsultas

- ***Exemplo (03):***
  - ***Fazendo uso*** da ***função*** **NVL** para ***converter valores nulos*** em ***valores não nulos***
  - **NVL** é ***utilizada*** para ***converter valores nulos*** de **ID_TIPO_PRODUTO** em **0** (*zero*)

```
SELECT id_tipo_produto, nm_tipo_produto
FROM tb_tipos_produtos
WHERE id_tipo_produto NOT IN (SELECT NVL(id_tipo_produto, 0)
                              FROM tb_produtos);
```

ID_TIPO_PRODUTO	NM_TIPO_PRODUTO
5	Magazine

Subconsultas Aninhadas

Subconsultas Aninhadas

- ***Permite aninhar subconsultas dentro*** de ***outras subconsultas*** até uma ***profundidade*** de **255**
- ***Essa técnica*** deve ser ***utilizada*** com ***moderação*** (o ***uso*** de ***junções*** de ***tabelas reduz*** o ***desempenho***)

Subconsultas Aninhadas

- ***Exemplo:***
  - ***Contém 3 consultas*** (***uma subconsulta aninhada***, ***uma subconsulta*** e a ***consulta externa***)

```
SELECT id_tipo_produto, AVG(preco)
FROM tb_produtos
GROUP BY id_tipo_produto
HAVING AVG(preco) < (SELECT MAX(AVG(preco))
                     FROM tb_produtos
                     WHERE id_produto IN (SELECT id_produto
                                          FROM tb_compras
                                          WHERE quantidade > 1)
                     GROUP BY id_tipo_produto)
ORDER BY id_tipo_produto;
```

ID_TIPO_PRODUTO	AVG(PRECO)
3	13.24
4	13.99
(null)	13.49

Subconsultas (Update / Delete)

Update e Delete contendo Subconsultas

- Em ***uma instrução*** **UPDATE**, podemos ***definir*** uma ***coluna*** com o ***resultado retornado*** por uma ***subconsulta*** de ***uma linha***
- ***Exemplo:***
  - ***Definir*** o ***salário*** do ***funcionário*** **nº 4** como a ***média*** dos ***tetos salariais retornados*** por uma ***subconsulta***

```
UPDATE tb_funcionarios
SET salario = (SELECT AVG(teto_salario)
               FROM tb_grades_salarios)
WHERE id_funcionario = 4;

ROLLBACK;
```

Update e Delete contendo Subconsultas

- ***Permite utilizar*** as ***linhas retornadas*** por uma ***consulta*** na ***cláusula*** **WHERE** de uma ***instrução*** **DELETE**
- ***Exemplo:***
  - ***Remover*** o(s) ***funcionário***(s) cujo ***valor salarial*** seja ***superior*** do que a ***média*** dos ***tetos salariais retornados*** por uma ***subconsulta***

```
DELETE
FROM tb_funcionarios
WHERE salario > (SELECT AVG(teto_salario)
                 FROM tb_grades_salarios);

ROLLBACK;
```

Exercícios

Using Subqueries

Exercícios

- ***Exercício (01):***
  - ***Write*** a ***query*** to ***display*** the ***sobrenome***, ***id_departamento***, and ***salario*** of ***any employee whose id_departamento*** and ***salario match*** the ***id_departamento*** and ***salario*** of ***any employee*** who ***earns*** a ***commission***.

Exercícios

- ***Exercício (02):***
  - ***Display*** the ***sobrenome***, ***nm_departamento***, and ***salario*** of ***any employee whose salario*** and ***percentual_comissao match*** the ***salario*** and ***percentual_comissao*** of ***any employee located*** in ***location*** **ID 1700**.

Exercícios

- ***Exercício (03):***
  - ***Create*** a ***query*** to ***display*** the ***sobrenome*** e ***salario*** for ***all employees*** who ***have*** the same ***salario*** and ***percentual_comissao*** as ***Kochhar***.
  - **Note:** *Do not display* ***Kochhar*** in the ***result set***.

Exercícios

- ***Exercício (04):***
  - ***Display details*** such as the ***id_empregado***, ***sobrenome*** e ***id_departamento*** of ***those employees*** who ***live*** in ***cities*** the ***names*** of which ***begin*** with ***T***.

Exercícios

- ***Exercício (05):***
  - ***Create*** a ***query*** to ***display*** the ***employees*** who ***earn*** a ***salario*** that is ***higher*** than the ***salary*** of ***all*** the ***sales managers*** (**ID_FUNCAO** = '**SA_MAN**'). ***Sort*** the ***results*** on ***salario*** from the ***highest*** to the ***lowest***.

Aula 03

Banco de Dados II

Departamento de Computação

ORACLE®

ACADEMY

Funções Simples

Funções Simples

- ***Explorar funções*** internas do ***Database Oracle***
- Uma ***função*** poderá ***aceitar*** **ZERO** ou ***mais parâmetros*** de ***entrada***, ***retornando*** um ***parâmetro*** de ***saída***
- ***Existem***, basicamente, ***dois tipos*** de ***funções*** no ***Oracle***
  - ***Funções*** de ***uma única linha***
  - ***Funções*** de ***Agregação***

Funções Simples

- ***Funções*** de uma ***única Linha:***
  - ***Operam*** uma ***linha*** por ***vez***, ***retornando*** uma ***linha*** de ***saída*** para ***cada linha*** de ***entrada***
  - ***Exemplo:***
    - **CONCAT(x, y):** ***retorna*** um ***string*** como ***resultado***
- ***Funções*** de ***Agregação:***
  - ***Operam*** sobre ***várias linhas*** por vez, ***retornando uma linha*** de ***saída***
  - ***Exemplo:***
    - **AVG(x):** ***retorna*** a ***média*** de "**x**", onde "**x**" pode ser uma ***coluna*** e ou ***uma expressão***

Usando Funções de uma única Linha

- ***Existem cinco tipos*** principais de ***funções*** de uma ***única linha:***

  1. ***Funções*** de ***Caractere***
     - ***Manipulam strings*** de ***caracteres***
  2. ***Funções Numéricas***
     - ***Efetuam cálculos***
  3. ***Funções*** de ***Conversão***
     - ***Convertem*** um ***valor*** de um ***tipo*** para ***outro***

Usando Funções de uma única Linha

- ***Existem cinco tipos*** principais de ***funções*** de uma ***única linha*** (*continuação*)***:***

  4. ***Funções*** de ***Data***

     - ***Processam datas*** e ***horas***

  5. ***Funções*** de ***Expressão Regular***

     - ***Utilizam expressões regulares*** para ***procurar dados***

Funções de Caracteres

- ***Aceitam entrada*** de ***caractere*** (***coluna*** e ou ***expressão***)
- **ACII(x):**
  - ***Obtém*** o ***valor*** **ASCII** do ***caractere*** "**x**"
- ***Exemplo:***

```
SELECT ASCII('a'), ASCII('A'), ASCII('z'), ASCII('Z'), ASCII('0'), ASCII('9')
FROM dual;
```

ASCII('A')	ASCII('A')_1	ASCII('Z')	ASCII('Z')_1	ASCII('0')	ASCII('9')
97	65	122	90	48	57

Funções de Caracteres

- **CHR(x):**
  - ***Obtém*** o ***caractere*** com o ***valor*** **ASCII**
- ***Exemplo:***

```
SELECT CHR(97), CHR(65), CHR(122), CHR(90), CHR(48), CHR(57)
FROM dual;
```

CHR(97)	CHR(65)	CHR(122)	CHR(90)	CHR(48)	CHR(57)
a	A	z	Z	0	9

Funções de Caracteres

- **CONCAT(x, y):**
  - ***Permite anexar*** (***concatenar***) "x" em "y" ***retornando*** um ***novo string***
- ***Exemplo:***
  - ***Concatenar nome*** com ***sobrenome*** do ***funcionário***

```
SELECT CONCAT(nome, sobrenome)
FROM tb_funcionarios;
```

CONCAT(NOME,SOBRENOME)
RobGreen
JaneBrown
JohnGrey
JeanBlue
HenryHeyson
KevinBlack
KeithLong
FrankHoward
DoreenPenn
JamesSmith
RonJohnson
FredHobbs
SusanJones

Funções de Caracteres

- **INITCAP**(x):
  - ***Converte*** a ***letra inicial*** de cada ***palavra*** de "x" em ***maiúsculo***
- ***Exemplo:***
  - ***Converte*** a ***primeira letra*** de ***cada palavra*** de **DS_PRODUTO** em ***maiúsculo***

```
SELECT id_produto, INITCAP(ds_produto)
FROM tb_produtos;
```

ID_PRODUTO	INITCAP(DS_PRODUTO)
6	Series On Mysterious Activities
7	Aliens Return
8	Adventures Of Heroes
9	Alien From Another Planet Lands On Earth
10	The Best Classical Music
11	The Best Popular Music
12	Debut Album
13	Their Greatest Hits
1	A Description Of Modern Science
2	Introduction To Chemistry
3	A Star Explodes
4	Action Movie About A Future War